SimRep: SIMION 과 같은 기능을 더 빠르게 계산하는 네이티브 이온 광학 시뮬레이터

SIMION_Rep 최종 기술 보고서

2026 년 10 월

저장소: SIMION_Rep (커밋 전 작업 트리 기준)

1. 개요

> **이 장에서 배울 것**
>
> 이 장에서는 SimRep 이 무엇을 하는 프로그램인지, SIMION 과 무엇을 같은 조건으로 비교했는지, 비교의 판정 규칙이 무엇인지 정리한다.

SIMION 은 전극이 만드는 전위를 격자 위에서 계산하고 그 전기장과 자기장 속에서 이온의 궤적을 적분하는 이온 광학 시뮬레이터다. 이 프로젝트의 출발점은 같은 일을 하는 프로그램을 새로 만들되 같은 문제에서 SIMION 보다 빠르고 정밀도는 같거나 더 높게 만드는 것이었다. 이 문서에서는 그 결과물을 SimRep 이라고 부른다. SimRep 은 Rust 로 쓴 계산 코어에 같은 코어를 부르는 명령행 도구, Python 패키지, Tauri 기반 데스크톱 앱을 붙인 구성이다. 목표와 범위는 docs/PLAN.md 의 1 절에 있다.

SIMION 의 방식을 그대로 옮기는 것은 목표가 아니었다. SIMION 은 전극을 격자 칸 단위의 계단으로 깎고 축차 과완화(SOR) 반복으로 전위를 풀며 전기장을 선형 보간으로 읽는다. 계단형 전극은 전극 근처의 전위를 틀리고 선형 보간은 전기장을 칸 경계에서 끊는다. SimRep 은 이 약점을 피하려고 전극의 실제 표면 위치를 반영하는 cut-cell 경계, 멀티그리드 솔버, 칸 경계에서도 매끄러운 전기장 재구성을 쓴다. 이 설계 결정도 docs/PLAN.md 의 3.1 절과 3.2 절에 근거를 두었다.

SIMION 프로그램 자체는 분석하지 않았다. 사용권 계약이 역분석을 금지하므로 SIMION 은 명령행 배치 실행 결과와 설치된 예제 파일, 공개 문서만 비교 기준(oracle)으로 삼았다. 비교에 쓴 장비와 소프트웨어는 표 1 과 같다.

항목	내용
CPU	AMD Ryzen 9 5950X (16 코어 32 스레드)
GPU	NVIDIA GeForce RTX 3070 8 GB (FP64 처리량은 FP32 의 1/64)
SIMION	SIMION-2024 (C:₩Program Files₩SIMION-2024), --nogui 배치 실행
운영체제	Windows + WSL2 Linux. GPU 실행은 Windows 네이티브 빌드
SimRep GPU API	wgpu (Vulkan, DX12). 셰이더는 WGSL

표 1 의 환경 항목은 docs/PLAN.md 의 2 절 표와 bench/simion-baseline.md 에 기록된 실행 환경을 옮긴 것이다. RTX 3070 에서는 배정밀도(FP64) 연산이 단정밀도의 1/64 로 느린데, 이것이 5 장에서 자기장 계산을 GPU 가 아니라 CPU 로 돌리는 이유다.

1.1 비교의 판정 규칙

속도 비교에는 두 가지 규칙을 두었다. 첫 번째 규칙은 정밀도를 맞춘 상태에서 비교한다는 것이다. SIMION 의 기본 수렴 설정은 엄격한 설정과 결과가 다르므로 전위 계산(refine)은 SIMION 이 허용하는 가장 엄격한 설정과 비교한다. 두 번째 규칙은 실제 계산 시간만 비교한다는 것이다. GPU 초기화, 파일 읽기, 결과 쓰기 같은 준비 시간은 판정에서 빼고 참고로만 보고한다. SIMION 쪽도 SIMION 이 스스로 보고하는 계산 시간을 쓴다. 두 규칙은 2026-10-23 의 사용자 결정으로 docs/PLAN.md 의 1 절에 있다.

정확도의 정답은 SIMION 의 결과가 아니다. SIMION 의 전극 근처 처리 방식은 공개되어 있지 않고 계단형 전극 자체가 오차를 만들기 때문이다. 그래서 정확도는 같은 문제를 훨씬 촘촘한 격자로 푼 기준해(보통 8 배 격자)와 비교해 판정하고 SIMION 의 결과도 같은 기준해와 비교한다. 이 규약 역시 docs/PLAN.md 의 1 절에 있다.

그림 1 은 완성된 앱의 실제 화면이다. einzel 렌즈 템플릿을 열고 전위를 푼 뒤 이온 1,000 개를 날린 상태다. 가운데 단면에서 색은 전위, 흰 선은 5 V 간격의 등전위선이다. 왼쪽에서 들어온 빔은 가운데 전극(80 V)을 지나며 오른쪽 한 점으로 모인다. 오른쪽 패널에는 전위 풀이가 40 회 반복 끝에 잔차 6.3e-11 로 수렴한 결과와 전극 전압 슬라이더가 있다. 이후 장에서는 이 화면 뒤의 계산을 차례로 설명한다.

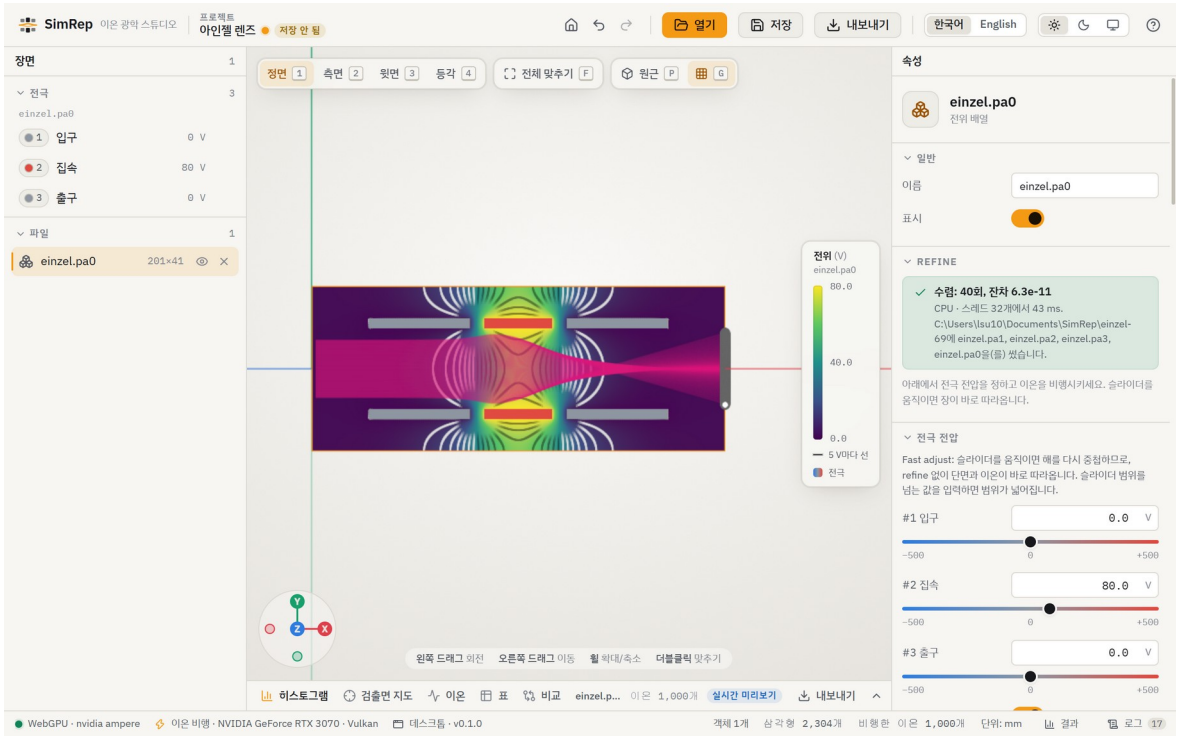


그림 1. SimRep 앱에서 einzel 렌즈 템플릿을 날린 화면(한국어, 밝은 테마). 출처: figures/F01.png, 캡처 조건은 figure-ledger.json 의 F01.

> **요점 정리**
>
> SimRep 은 SIMION 의 핵심 기능을 하는 네이티브 프로그램이다. 비교는 정밀도를 맞춘 뒤 계산 시간만으로 하고 정확도의 정답은 촘촘한 격자의 기준해로 삼는다. SIMION 프로그램은 분석하지 않았고 실행 결과와 공개 문서만 썼다.

2. 프로그램 구성

> **이 장에서 배울 것**
>
> SimRep 을 이루는 부분과 사용자가 형상에서 결과까지 가는 처리 흐름을 정리한다.

SimRep 의 계산 코어는 역할별로 나눈 11 개의 Rust 크레이트다. 크레이트는 Rust 의 컴파일 단위이자 라이브러리 단위다. 표 2 는 각 크레이트의 역할이다. 같은 코어를 명령행 도구 simrep, Python 패키지 simrep, 데스크톱 앱이 함께 쓴다. 그래서 입력, 형상, 적분기와 정밀도 설정을 맞추면 세 경로는 같은 물리 구현으로 계산한다. 다만 앱의 실시간 미리보기는 GPU 단정밀도, 정밀 모드는 CPU 배정밀도를 쓰므로 고른 설정에 따라 결과가 달라질 수 있다.

크레이트	역할	근거
simrep-core	격자, 전위 배열(PA), 공용 자료형	Cargo.toml description
simrep-solve	전위 솔버: CPU/GPU 멀티그리드, cut-cell, 유전체, 자기정역학	Cargo.toml description, docs/dielectric.md 의 Discretization and interfaces 절, docs/magnetostatics.md 의 Equations and flight contract 절
simrep-fly	전기장 재구성, 적분기, 이온 비행(CPU/GPU), 자기장, 충돌, 공간전하	Cargo.toml description, docs/physics-magnetic-collision.md 의 Units and magnetic fields·HS1·SDS 절, docs/physics-space-charge.md 의 Direct Coulomb·PIC and Poisson 절
simrep-io	SIMION 파일 형식 (.pa, .fly2, .gem, .rec)과 STL	Cargo.toml description
simrep-gem	SIMION GEM 구성 형상	Cargo.toml description

	해석과 PA 래스터화	
simrep-lua	SIMION 입력 형식이 함께 쓰는 제한된 Lua 5.1 실행 환경	Cargo.toml description
simrep-geom	부호 거리 함수(SDF)로 나타낸 전극 형상	Cargo.toml description
simrep-mesh	삼각형 메시 가져오기, 복셀화, SDF, 과학 데이터 내보내기	Cargo.toml description
simrep-cad	선택 기능: OpenCascade 로 STEP/IGES 전극 가져오기	Cargo.toml description, docs/step-import.md 의 Kernel decision 절
simrep-cli	명령행 도구 simrep	Cargo.toml description
simrep-py	NumPy 기반 Python 패키지	Cargo.toml description

표 2 의 역할 설명은 각 크레이트 Cargo.toml(예: crates/simrep-core/Cargo.toml)의 description 항목과 근거 열의 구현 문서 절을 바탕으로 이 문서에서 요약한 것이다. 이 가운데 simrep-solve 와 simrep-fly 가 계산의 대부분을 맡고, 나머지는 입력 형식과 사용자 접점이다.

사용자는 세 가지 방법으로 SimRep 을 쓴다. 명령행 도구 simrep 은 refine, refine-geometry, fly, gem2pa, mesh-import, export 같은 하위 명령으로 일괄 작업과 검증을 맡고 Python 패키지 simrep 은 같은 기능을 NumPy 배열로 제공해 매개변수 탐색과 분석에 쓰인다. 명령행 기능과 Python 기능의 대응은 docs/python.md 의 CLI parity 절에 표로 정리해 두었고 데스크톱 앱은 7 장에서 다룬다.

> **요점 정리**
>
> 계산 코어는 11 개 Rust 크레이트이고, 명령행 도구, Python 패키지, 데스크톱 앱이 같은 코어를 쓴다. 그래서 같은 설정에서는 세 경로가 같은 계산 구현을 쓴다.

3. 전위 계산

> **이 장에서 배울 것**
>
> 전극 전위를 격자 위에서 푸는 방법, SIMION 의 계단형 전극이 왜 오차를 만드는지, 그리고 cut-cell 경계와 멀티그리드가 그 오차와 계산 시간을 어떻게 줄이는지 배운다.

전위 계산은 refine 이라고 부르는 단계다. 전극의 전압이 정해지면 전극 사이 진공의 전위는 라플라스 방정식을 만족한다. SimRep 은 이 방정식을 균일 격자의 2 차 중심차분으로 바꾼다. 식 (1)이 그 이산 연산자다.

$$L_h V_i = \sum_a \frac{V_{i-e_a} - 2V_i + V_{i+e_a}}{h_a^2}, \quad -L_h V = \rho/\varepsilon_0 \quad (1)$$

식 (1)의 L_h 는 이웃한 격자점 전위로 전위의 곡률을 근사하는 연산자다. V_i 는 격자점 i 의 전위(V), e_a 는 축 a 방향으로 한 칸 옮기는 기호, h_a 는 그 축의 격자 간격(mm)이다. 공간전하가 없으면 오른쪽의 ρ/ε_0 가 0 이 되어 라플라스 방정식이 되고 공간전하가 있으면 같은 연산자로 푸아송 방정식을 푼다. ρ 는 전하 밀도(C/mm³)다. ε_0 는 mm 단위로 쓴 진공 유전율 8.8541878128e-15 F/mm 다. 원통 대칭 배열에서는 반지름 방향 항에 1/r 보정이 붙고 축 위의 점은 $4[V(h_r) - V(0)]/h_r^2$ 로 처리한다. 이 연산자와 원통 처리는 docs/cpu-solver.md 의 Discrete operator 절에 정의되어 있다. 구현은 crates/simrep-solve/src/mg.rs 의 solve 함수다. 공간전하가 있으면 solve_poisson 함수를 쓴다. 표준 정전기학의 라플라스·푸아송 방정식을 이 문서의 기호로 옮겼다.

3.1 멀티그리드와 수렴 판정

격자점이 수백만 개면 식 (1)을 반복법으로 푸는 시간이 계산의 대부분을 차지한다. SIMION 은 축차 과완화(SOR) 반복을 쓰는데 이 방법은 긴 파장의 오차를 천천히 줄인다. SimRep 은 갤러킨(Galerkin) 멀티그리드를 쓴다. 멀티그리드는 촘촘한 격자에서 짧은 파장의 오차를 지우고 남은 긴 파장의 오차는 거친 격자로 옮겨 빠르게 지우는 방법이다. 거친 격자 연산자는 촘촘한 연산자를 보간 행렬 P 로 끼워 $P^T A P$ 로 만든다. 이 때문에 거친 격자도 같은 에너지를 유지하고 가는 전극이 거친 격자에서 사라지지 않도록 전극 점을 보존하며 격자를 줄인다. CPU 경로는 모든 계산을 배정밀도(f64)로 하고 평활에는 체비쇼프-야코비 반복을 앞뒤 세 번씩 쓴다. 이 구성은 docs/cpu-solver.md 의 Coarsening electrodes 절에 있다.

수렴 판정은 SIMION 과 다르다. SIMION 은 반복 사이에 전위의 최대 변화량이 기준보다 작아지면 멈춘다. 변화량이 작다는 것이 방정식이 풀렸다는 뜻은 아니므로 SimRep 은 방정식의 잔차를 직접 잰다. 식 (2)가 그 판정이다.

$$\|r\|_\infty \le \tau_{abs} + \tau_{rel} s \quad (2)$$

식 (2)의 r 은 자유 격자점에서 계산한 물리 잔차, 즉 식 (1)의 왼쪽과 오른쪽의 차이이고 단위는 V/mm² 다. s 는 상대 허용값의 기준 크기다. 전극 전압을 첫 고정 전압만큼 옮겨 만든 오른쪽 항 b 의 최대 노름을 쓴다. 이 오른쪽 항이 0 이면 단위 전압 스텐실의 크기를 쓴다. τabs 는 절대 허용값(V/mm²), τrel 은 무차원 상대 허용값이다. 기본 상대 허용값은 1e-10 이다. 이 판정은 docs/cpu-solver.md 의 Convergence and reporting 절에 정의되어 있다. 구현은 crates/simrep-

solve/src/mg.rs 의 reached 함수다. 잔차가 작다는 것은 이산 방정식이 풀렸다는 뜻이다. 격자가 실제 전극 모양을 얼마나 잘 나타내는지는 별개의 문제다. 그 오차는 3.3 절에서 다룬다.

GPU 경로는 같은 멀티그리드를 단정밀도(f32) 보정 사이클로 돌리고 바깥에서 배정밀도 잔차로 보정하는 반복 개선(iterative refinement)을 쓴다. 덕분에 GPU 의 빠른 단정밀도 연산을 쓰면서도 최종 잔차는 식 (2)의 배정밀도 기준을 만족한다. 이 방식은 docs/gpu-solver.md 의 Numerical method 절에 정리되어 있다.

멀티그리드가 실제로 몇 번의 사이클로 수렴하는지는 그림 2 에 나와 있다. 가로축은 V-사이클 번호, 세로축은 식 (2)의 상대 잔차(로그축)다. 두 곡선 모두 사이클마다 거의 같은 비율로 줄어드는 직선이다. 격자점 16 만 개인 4 중극은 13 회, 1,737 만 개인 큰 배열은 11 회 만에 상대 잔차 1e-12 에 닿았다. 격자가 100 배 커져도 사이클 수가 늘지 않는 것이 멀티그리드의 특징이다.

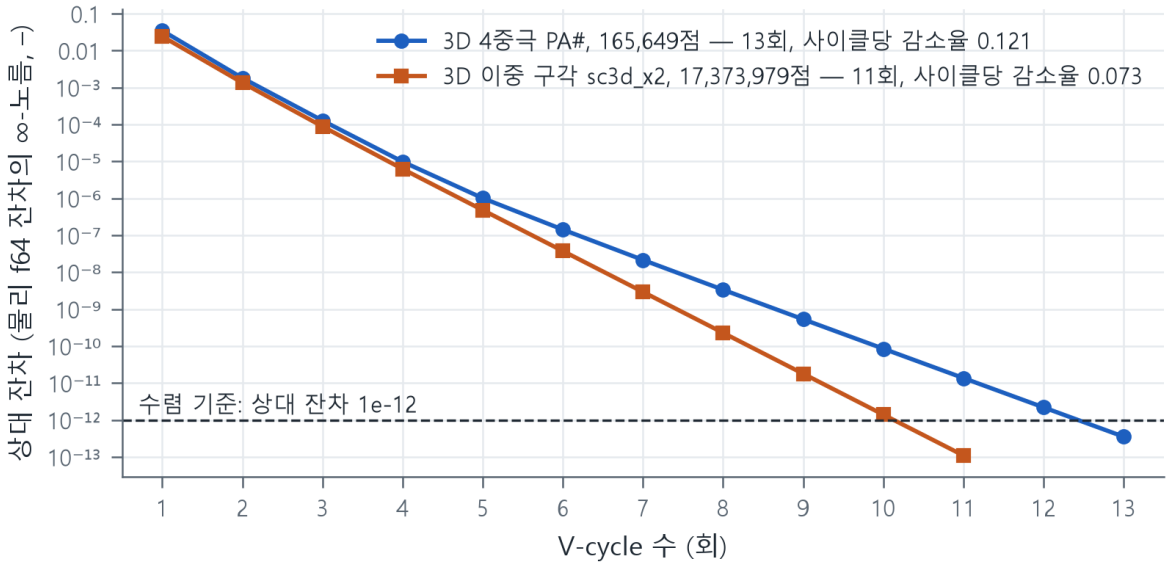


그림 2. 멀티그리드 V-사이클별 상대 잔차. 현재 빌드의 simrep refine 을 실행해 기록한 잔차 이력을 그렸다. 실행 명령은 figure-ledger.json 의 F05. 출처: figures/F05.png.

3.2 실제 SIMION 배열에서의 비교

속도와 정확도를 함께 보기 위해 큰 3D 배열(259×259×259, 약 1,737 만 격자점)을 두 프로그램으로 풀었다. SIMION 설치 예제의 examples/performance_tests 에는 큰 3D 배열이 없어서, 두 구 껍질로 된 GEM 파일을 SIMION 의 gem2pa 로 만든 sc3d_x2 배열을 썼다(bench/simion-baseline.md 의 Large 3D refine case 절). SIMION 은 허용되는 가장 엄격한 수렴 기준 1e-7 로, SimRep 은 잔차 1e-12 로 풀었다. 표 3 이 계산 시간이다.

계산	준비(ms)	풀이(ms, 전송 포함)	SIMION 대비
SimRep CPU 16 코어	2376	5002	풀이만 2.8 배
SimRep GPU (Vulkan)	829	736	풀이만 19.3 배, 준비

			포함 9.1 배
SIMION 엄격 설정	구분 안 됨	14230 (SIMION 보고 계산 시간)	기준

표 3 의 SimRep 행은 bench/refine-real.md 의 End-to-end performance 절, SIMION 행은 bench/simion-baseline.md 의 Refine 절에서 가져왔고 배수는 이 문서에서 두 값을 나눈 것이다. GPU 풀이 시간에는 데이터 전송이 들어 있어 순수 커널 시간보다 크다. 그래도 준비 시간을 포함해 계산하면 GPU 는 SIMION 보다 9 배 빠르다. CPU 는 풀이만 놓고 보면 2.8 배 빠르다. 준비 시간을 더하면 차이는 1.9 배로 줄어든다. 정확도는 모든 격자점에서 엄격 설정 SIMION 과 최대 6.4e-5 V 이내로 같았다(bench/refine-real.md 의 Large-array tight reference and CPU/GPU agreement 절). 반면 SIMION 의 기본 설정은 엄격 설정과 최대 0.92 V 가 달랐다. 이 차이 때문에 1 장의 규칙대로 엄격 설정을 비교 기준으로 삼았다.

3.3 계단형 전극과 cut-cell 경계

격자 계산의 가장 큰 오차는 전극 모양에서 나온다. SIMION 은 격자점이 전극 안에 있는지 밖에 있는지만 정하므로 매끄러운 전극 표면이 격자 칸 크기의 계단으로 바뀐다. 이 계단이 전극 근처의 전위를 틀리게 하고 이 오차는 격자를 키워도 느리게 줄어든다. SimRep 은 전극 표면이 격자 칸 사이 어디를 지나는지 계산해 그 위치를 연산자에 넣는다. 이 방식을 cut-cell 경계라고 부른다. 식 (3)이 그 핵심이다.

$$c\,(V_j - V_i) \longrightarrow \frac{c}{\theta}\,(V_\Gamma - V_i) \quad (3)$$

식 (3)은 Gibou et al., 2002 의 식 (21)인 선형 대칭 ghost-fluid 이산화를 이 문서의 기호로 옮긴 것이다. 진공 점 i 와 전극 안쪽 점 j 를 잇는 연결에서 원래 계수 c 로 V_j 를 쓰던 항을 표면까지의 비율 θ(0 보다 크고 1 이하)로 나눈 계수와 표면 전위 V_Γ 로 바꾼다. 표면이 점 i 에 가까울수록 θ 가 작아지고 표면 전위의 영향이 커진다. 진공 점끼리의 연결은 바꾸지 않으므로 행렬이 대칭 양의 정부호로 남고 멀티그리드를 그대로 쓸 수 있다. 같은 논문의 비대칭 2 차 공식(식 (20))은 이 성질을 잃기 때문에 쓰지 않았다. 표면이 격자점에 아주 가까울 때는 그 점을 전극 전압으로 고정한다. 이 처리와 선택 이유는 docs/cutcell.md 의 Operator and symmetry invariant 절에 있고 구현은 crates/simrep-solve/src/cutcell.rs 의 CutRow 다.

그림 3 은 4 중극의 4 분의 1 단면을 두 경계 처리로 푼 실제 계산 결과다. 색은 전위, 회색은 전극, 점선은 GEM 파일에 정의된 반지름 5.75 mm 의 원래 막대 표면이다. 왼쪽 계단형에서는 회색 칸이 점선 안쪽을 계단으로 채워 전극 표면이 칸 크기만큼 어긋나 있다. 오른쪽 cut-cell 에서는 표면 교차점(주황 점)이 점선 위에 정확히 놓인다. 그래서 전극 바로 옆 등전위선 모양도 두 그림에서 다르다. 이 차이가 4 장의 이온 도착 위치 오차로 이어진다.

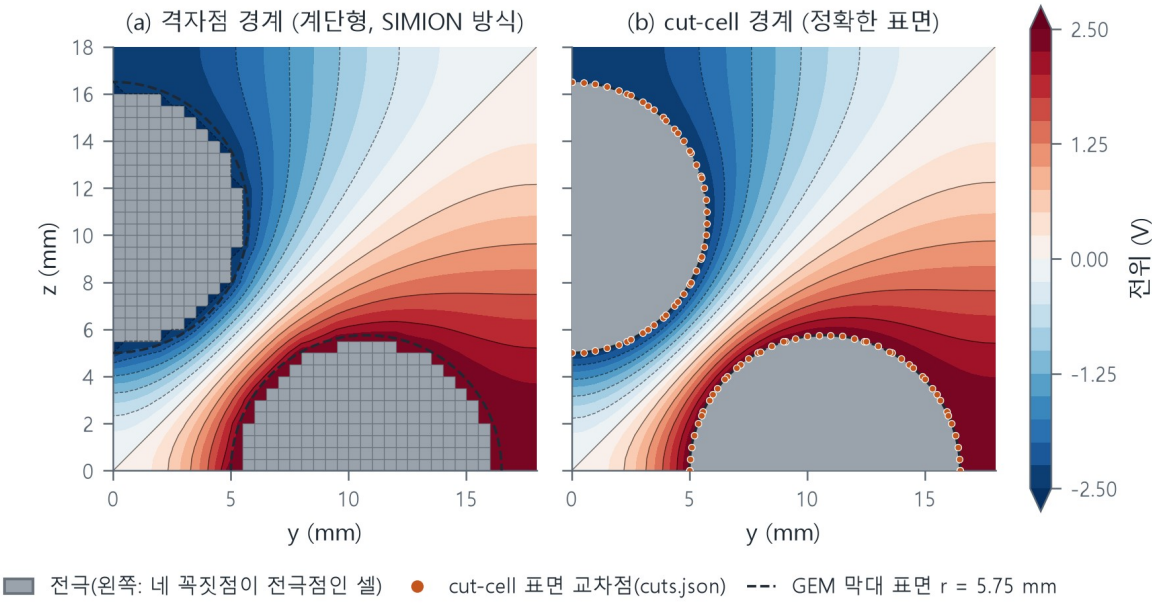


그림 3. 4 중극 4 분의 1 단면의 전위: 계단형 경계(왼쪽)와 cut-cell 경계(오른쪽). 현재 빌드의 simrep 으로 같은 형상을 두 방식으로 푼 결과를 그렸다. 출처: figures/F04.png, 제작 기록은 figure-ledger.json 의 F04.

cut-cell 의 효과는 반지름 0.31 과 0.83(영역 [−1, 1]의 단위 길이)인 두 구 전극(1 V, 0 V)처럼 정확한 해를 아는 문제로 확인했다. 표 4 는 같은 격자에서 두 경계 처리의 최대 전위 오차와 계산 시간이다.

장치·격자	cut-cell 계산 (ms)	계단형 계산(ms)	cut-cell 최대 오차(V)	계단형 최대 오차(V)
CPU 128³	238.4	219.1	6.71e-4	3.99e-2
CPU 256³	2228.0	2000.8	2.68e-4	2.57e-2
GPU 128³	89.5	64.7	6.71e-4	3.99e-2
GPU 256³	285.5	218.0	2.68e-4	2.57e-2

표 4 의 값은 docs/cutcell.md 의 Measured acceptance 절에서 가져왔다. cut-cell 은 계산 시간이 약 9~38% 늘어난다. 대신 최대 오차가 약 60~100 배 작다. 같은 메모리 예산으로 비교해도 결과는 같았다. 계단형 격자를 같은 메모리 예산 안에서 가장 크게(129³, 257³) 잡아도 오차가 cut-cell 보다 컸다. 128³ 예산에서는 77.9 배, 256³ 예산에서는 83.7 배였다. 이 비교는 GPU 버퍼 메모리를 기준으로 했고 docs/cutcell.md 의 Equal memory budget 절에 있다. 원래 계획의 3 단계 목표는 같은 메모리에서 오차 10 배 감소였으므로 이 목표를 넘었다.

그림 4 는 격자를 촘촘하게 할 때 오차가 줄어드는 속도다. 두 패널 모두 가로축은 단위 길이로 잰 격자 간격 h 이고 두 축은 로그다. 왼쪽 세로축은 최대 전위 오차(V), 오른쪽 세로축은 표면에서 두 칸 안의 최대 전기장 성분 오차(V/단위 길이)다. cut-cell 의 전위 오차는 기울기 약 2, 즉 2 차로 줄어든다(구 2.21, 평면 원통 1.92, 축대칭 원통 1.99). 전극 표면 바로 옆의 전기장

오차는 약 1 차로 줄어든다. 그래서 전위는 2 차 정확도라고 말할 수 있지만, 표면 근처 전기장까지 2 차라고 말할 수는 없다.

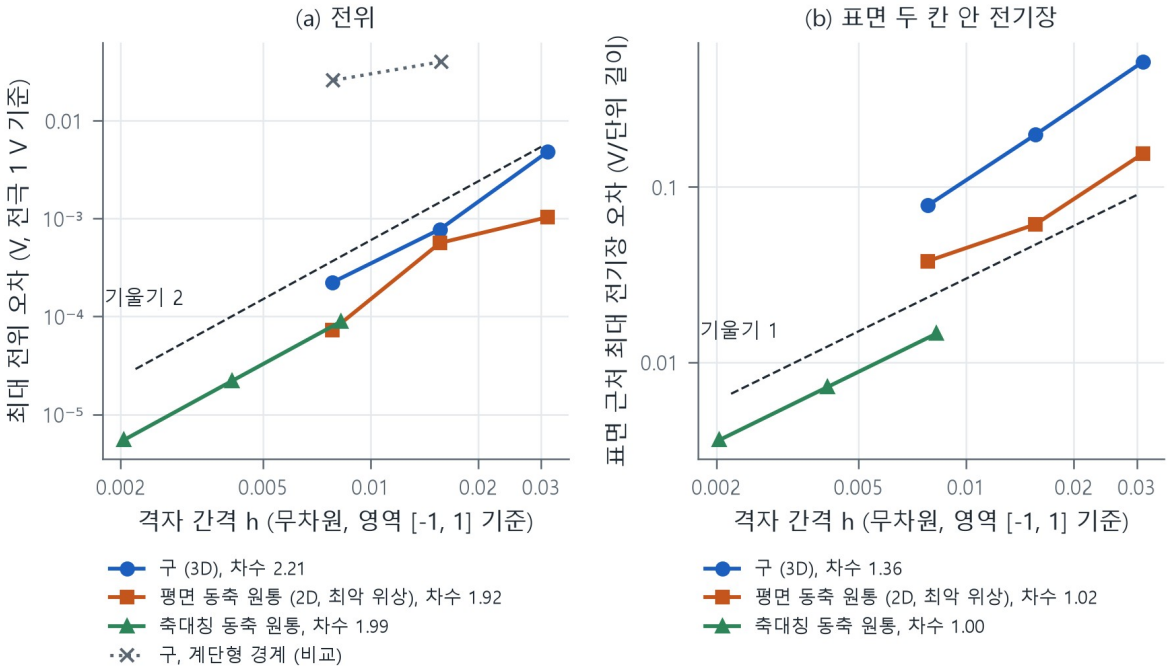


그림 4. cut-cell 경계의 격자 수렴. 왼쪽은 최대 전위 오차, 오른쪽은 표면 두 칸 안의 최대 전기장 성분 오차다(가로축 무차원 격자 간격, 로그-로그). 값은 docs/cutcell.md 의 Grid convergence 절에 기록된 측정값을 이 문서에서 다시 그린 것이다. 출처: figures/F06.png.

> **요점 정리**
>
> SimRep 은 식 (1)을 갤러킨 멀티그리드로 풀고, 반복 변화량이 아닌 잔차로 수렴을 판정한다. 1,737 만 점 배열에서 GPU 풀이는 엄격 설정 SIMION 보다 19 배(준비 포함 9 배) 빠르고 결과는 6.4e-5 V 안에서 같다(표 3). cut-cell 경계는 같은 메모리에서 계단형보다 전위 오차를 약 80 배 줄인다(docs/cutcell.md 의 Equal memory budget 절).

4. 이온 비행

> **이 장에서 배울 것**
>
> 이온의 운동방정식과 적분기, 격자 위 전위에서 매끄러운 전기장을 만드는 방법, 전극 충돌 판정을 정리하고 einzel 렌즈와 4 중극에서 SIMION 과 정확도·계산 시간을 비교한다.

전위가 풀리면 이온을 날린다. 이온은 전기장과 자기장이 주는 로런츠 힘을 받으며 움직인다. SimRep 은 속도 대신 고유 속도 u = γv 를 상태로 삼아 상대론 효과를 포함한다. 운동방정식은 식 (4)다.

$$\frac{d\boldsymbol{x}}{dt}=\boldsymbol{v}=\frac{\boldsymbol{u}}{\gamma},\gamma=\sqrt{1+|\boldsymbol{u}|^2/c^2},\frac{d\boldsymbol{u}}{dt}=\frac{q}{m}\left[a_E\boldsymbol{E}+a_B(\boldsymbol{v}\times\boldsymbol{B})\right] \quad (4)$$

식 (4)의 x 는 위치(mm), t 는 시간(μs), v 는 속도(mm/μs), E 는 전기장(V/mm), B 는 자기장(G)이다. q 는 기본 전하 단위의 전하수, m 은 원자질량단위(amu)의 질량이다. aE 와 aB 는 이 단위들을 SI 로 맞추는 상수다. aE 는 기본 전하를 원자질량단위로 나누고 10^{-6} 을 곱한 값이고 aB 는 그 10^{-4} 배다. 이온이 빛보다 훨씬 느리면 γ 는 1 에 가까워지고 식은 뉴턴 운동방정식이 된다. c 는 빛의 속도 299792.458 mm/μs 이고 γ 는 무차원이다. 이 식은 상대론적 로런츠 힘 운동방정식(Jackson, 1999 의 12 장)을 이 문서의 단위로 옮긴 것이다. 구현은 crates/simrep-fly/src/flight.rs 의 derivative 함수다. 상수 aE, aB, c 는 crates/simrep-fly/src/lib.rs 에 정의되어 있다. CLI 는 상대론 모드를 기본으로 켠다.

4.1 적분기와 스텝 크기

정전기장에서는 Dormand-Prince 5(4) 방식의 RK45 적응 적분기를 쓴다. 이 방식은 한 스텝에서 5 차와 4 차 해를 함께 계산하고, 둘의 차이를 오차 추정으로 써서 다음 스텝 크기를 정한다. 기본 적분기는 자기장이 있어도 RK45 이고, 사용자가 Boris 적분기를 고를 수도 있다. Boris 방식은 균일 자기장에서 속력과 에너지를 반올림 오차 수준까지 보존하지만 회전 위상에는 2 차 오차가 남는다. 5 장의 자기장 비교(표 8)는 RK45 로 했다. 두 적분기는 crates/simrep-fly/src/flight.rs 의 rk45, boris 함수에 구현되어 있다.

스텝 크기에는 오차 추정과 별도로 물리적 상한을 둔다. 한 스텝에 가장 작은 격자 칸 4 개 이상을 건너지 않고 자기장 속 회전각과 시간 가변 전압의 위상 변화가 한 스텝에 0.2 rad 을 넘지 않는다. 허용 오차는 1e-12 에서 시작하고 정전기 문제에서는 출발점 근처 전기장의 변화로부터 1e-12 와 1e-8 사이의 값을 고른다. 이 기본값과 측정 근거는 bench/default-flight.md 의 Policy and tolerance selection 절에 있다.

4.2 매끄러운 전기장 재구성

SIMION 은 격자 전위를 선형 보간해 전기장을 읽는다. 선형 보간한 전기장은 격자 칸 경계에서 끊어진다. 적응 적분기는 이 끊김을 큰 오차로 보고 스텝을 줄이므로 정확도와 속도가 함께 떨어진다. SimRep 은 격자점마다 전위의 1 차·2 차 도함수를 추정한 다음 칸 안을 5 차 에르미트 다항식으로 채운다. 그 결과 전위는 2 번 미분할 때까지 연속(C2)이고 전기장은 칸 경계에서도 연속(C1)이며 격자점에서는 원래 전위값을 정확히 지난다. 전극 근처에서는 진공 쪽 점만으로 도함수를 추정해 전극 안쪽 값이 섞이지 않게 한다. 이 경계 인식 재구성은 bench/fly-real.md 의 Boundary-aware correction results 절에 정리되어 있고 구현은 crates/simrep-fly/src/field/boundary.rs 다.

4.3 전극 충돌과 정확한 표면

이온이 전극에 닿았는지는 두 가지 방식으로 판정한다. 기본 방식은 격자 칸의 꼭짓점(2D 에서 4 개, 3D 에서 8 개)이 모두 전극인 칸에 들어가면 충돌로 본다. 이 방식은 SIMION 의 전극 칸

판정과 같은 계단형 모양을 쓴다. 더 정확한 방식은 정확한 표면 파일 (.cuts.json)을 쓴다. 형상을 refine-geometry 로 풀면 원래의 GEM 구성 형상이나 메시 삼각형을 이 파일에 함께 저장하고 비행 때 이온 경로와 실제 표면의 교점을 직접 계산한다. 이 경로는 현재 CPU 에서만 돈다. 형식과 사용법은 docs/geometry-pipeline.md 의 Outputs and flying 절에 있다.

4.4 정확도 판정 기준

정확도는 이온 한 개마다 도착 위치를 기준해와 비교하고 그 차이의 제곱평균제곱근(RMS)으로 요약한다. 식 (5)가 이 문서의 판정 규약이다.

$$RMS = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \left| \boldsymbol{x}_i - \boldsymbol{x}_i^{ref} \right|^2} \quad (5)$$

식 (5)의 x_i 는 i 번째 이온의 도착 위치(mm), x_i^{ref} 는 같은 이온을 8 배 촘촘한 격자에서 날린 기준해의 도착 위치, N 은 이온 수다. 이 값은 이 문서의 판정 규약이며, 같은 정의를 SIMION 의 결과에도 그대로 적용했다. 도착한 면이 기준해와 달라진 이온 수도 함께 센다.

4.5 einzel 렌즈

einzel 렌즈는 세 원통 전극(0 V, 110 V, 0 V)으로 이온 빔을 모으는 렌즈다. SIMION 예제의 91×20 원통 대칭 배열에 질량 100 amu, 에너지 200 eV 의 이온을 반지름 −15~15 mm 에 고르게 놓고 날렸다. 그림 5 는 이와 별도로 앱의 einzel 템플릿(201×41 격자, 간격 0.5 mm, 전극 0/80/0 V, 100 eV)을 SimRep 앱에서 날린 실제 화면이다. 위쪽 단면 뷰에서 궤적이 가운데 전극을 지나며 한 점으로 모인다. 아래 TOF 히스토그램에서는 이온 1 만 개의 비행시간이 가장 이른 8.14 μs 근처에 몰리고 뒤로 긴 꼬리가 이어진다. 오른쪽 상자의 질량 분해능 271 은 이 분포의 반치폭에서 앱이 계산한 값이다.

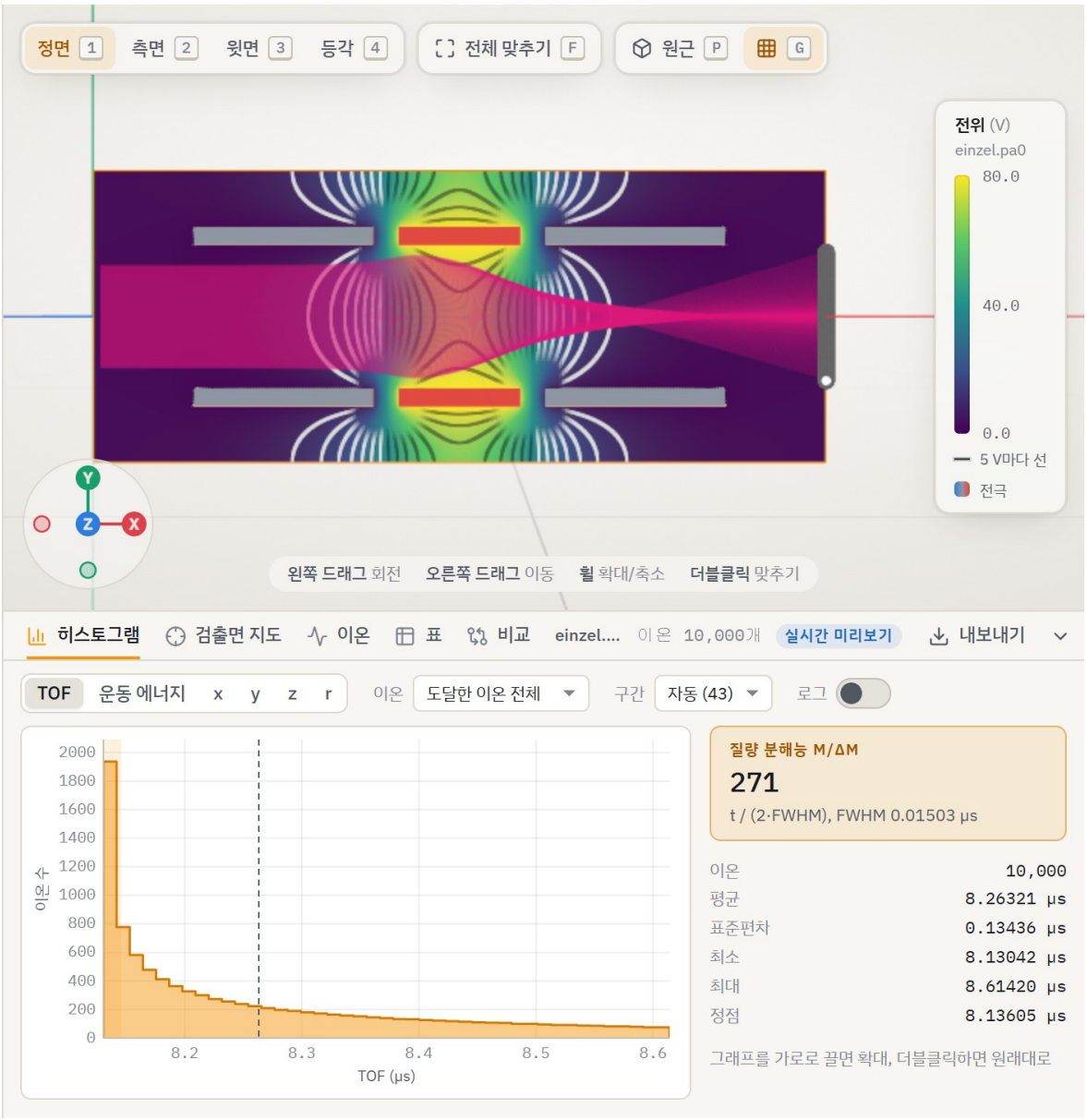


그림 5. 앱의 einzel 렌즈 템플릿을 날린 화면: 전위 단면과 궤적, 비행시간(TOF) 히스토그램. 출처: figures/F07.png, 캡처 조건은 figure-ledger.json 의 F07.

표 5 는 같은 1,000 개 이온의 정확도와 10,000 개 이온의 계산 시간이다. 기준해는 8 배 격자다.

방법	도착 위치 RMS (mm)	최대 (mm)	계산 시간, 이온 1 만 개 (s)
SIMION, T.Qual 209	1.106	4.282	—
SIMION, 기본 품질	—	—	2.13
SimRep CPU 16 코어, 기본 설정	0.226	1.323	0.185

SimRep GPU, 배정밀도	0.226	1.324	—

표 5 의 정확도는 bench/fly-real.md 의 Boundary-aware correction results 절과 bench/default-flight.md 의 Main integration 절, GPU 행은 bench/fly-gpu.md 의 Native precision study 절, SIMION 계산 시간은 bench/simion-baseline.md 의 Fly 절에서 가져왔다. SimRep CPU 의 정확도와 계산 시간은 현재 기본 설정(four-cell, bench/default-flight.md 의 Main integration 절)의 값이고, GPU 행은 이와 별도로 보존한 1,000 개 정확도 비교 설정의 값이다. 따라서 CPU 와 GPU 행은 같은 설정에서의 직접 비교가 아니다. SimRep 의 오차는 SIMION 이 더 비싼 T.Qual 209 로 계산한 오차보다 4.9 배 작다. 계산 시간은 SIMION 이 더 빠른 기본 품질로 계산한 시간과 비교해도 11.5 배 짧다. 즉 정확도가 높은 쪽이 더 빠르므로 정밀도를 맞추면 차이는 더 커진다. 같은 1,000 개를 SIMION T.Qual 209 로 날린 10.39 s 와, 비교용으로 보존한 예전 one-cell 설정의 SimRep 0.224 s 를 나누면 46 배다(bench/default-flight.md 의 Retained one-cell baseline 절). 이 오차 감소는 4.2 절의 매끄러운 전기장 재구성에서 나온다. 같은 격자에서 예전의 단순 스플라인 재구성은 RMS 5.90 mm 였다.

4.6 3D 4 중극

4 중극은 원통 전극 네 개가 만드는 3D 문제다. SIMION 예제 형상을 121×37×37 격자(간격 0.5 mm)로 풀고 전극에 2.5 V, −2.5 V, 0 V 를 걸어 이온 1,000 개를 날렸다. 기준해는 같은 실제 형상을 8 배 격자로 푼 결과다. 그림 6 은 이온별 도착 위치 오차의 누적 분포다. 가로축은 이온 하나의 오차(mm, 로그축), 세로축은 그 오차 이하인 이온의 비율이다. 곡선이 왼쪽에 있을수록 정확하다. SIMION(주황)은 대부분의 이온 오차가 0.1~0.3 mm 이고 약 10%는 수 mm 에 이른다. 같은 cut-cell 전위장에서 충돌 판정만 계단형으로 한 SimRep(회색)은 대부분 0.002 mm 근처지만 출구 근처에서 전극을 다르게 판정한 이온 몇 %가 수 mm 오차를 낸다. 정확한 표면 경로(파랑)는 최대 오차가 약 0.0202 mm 다. 즉 전위장과 충돌 판정이 모두 실제 표면을 따라야 오차가 크게 줄어든다.

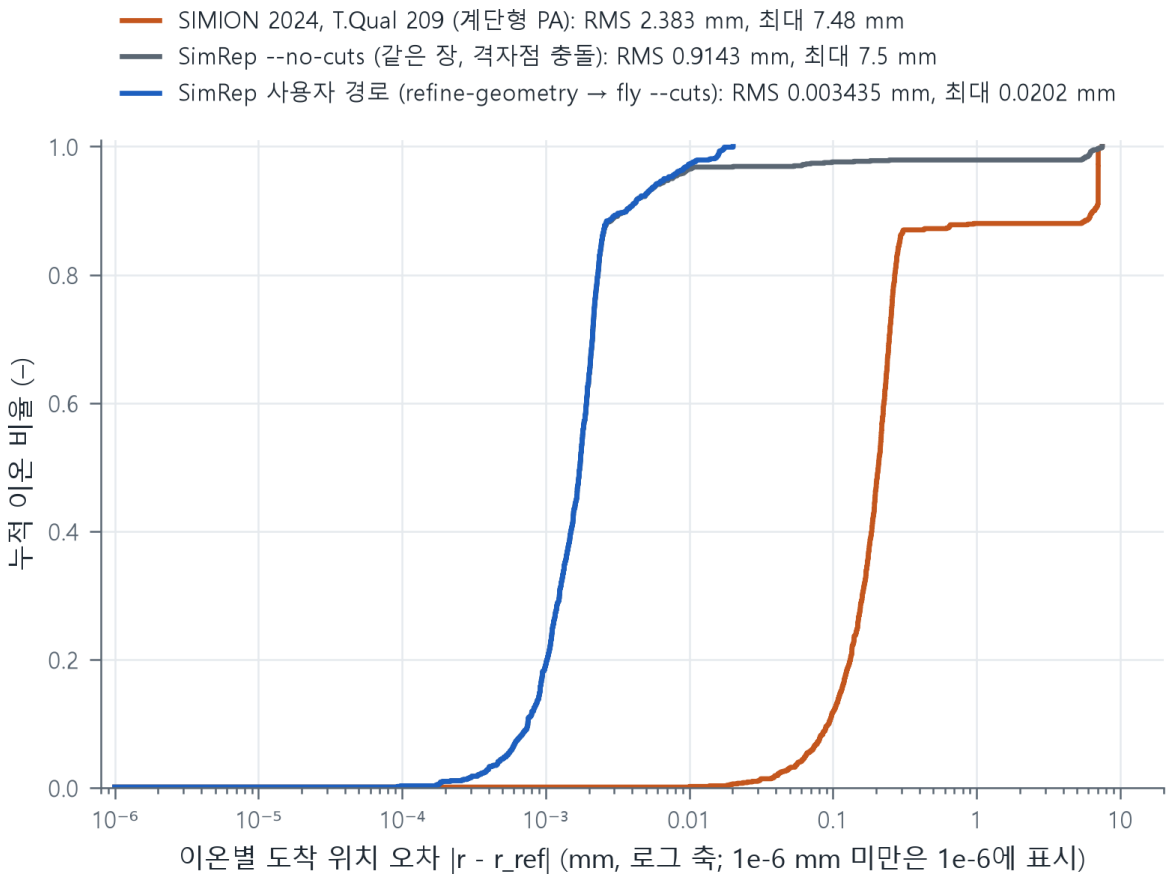


그림 6. 3D 4 중극에서 이온별 도착 위치 오차의 누적 분포(실제 형상 8 배 격자 기준). SIMION T.Qual 209, 같은 장에서 계단형 충돌 판정을 한 SimRep, SimRep 정확한 표면 경로(refine-geometry → fly --cuts)를 비교했다. 현재 빌드로 실행한 결과를 그렸고 명령은 figure-ledger.json 의 F08. 출처: figures/F08.png.

경로	도착 위치 RMS (mm)	출구가 바뀐 이온	계산 시간 (s)
SIMION T.Qual 209	2.3826	—	9.28
SimRep 계단형 배열, CPU	2.3828	—	0.0658
SimRep 정확한 표면 경로, CPU	0.0034	0	0.1308

표 6 의 값은 bench/fly-real.md 의 Compute time at achieved precision 절과 docs/geometry-pipeline.md 의 V3 compute overhead and verification 절에서 가져왔다. SimRep 의 계단형 경로는 SIMION 과 같은 오차(2.38 mm)를 141 배 짧은 시간에 낸다. 정확한 표면 경로는 오차를 0.0034 mm 로 약 700 배 줄이면서도 SIMION 보다 71 배 빨랐다. 다만 이 정확도 차이는 보간만의 효과가 아니다. 형상, 전위 풀이, 충돌 판정이 모두 실제 표면을 쓰므로 그렇다. 정확한 표면 경로의 시간은 같은 cut-cell 전위장에서 충돌 판정만 계단형으로 한 경로(0.0977 s)보다 1.34 배 길었다. 표 6 의 계단형 배열 경로(0.0658 s)와 비교하면 1.99 배다. 이후 먼 전극 구간을

거리 계산 한 번으로 제외하도록 고쳤다. 그 뒤에는 4 중극 0.98 배, einzel 1.04 배로 계단형과 거의 같아졌고 각 경우 7 회 반복 실행의 도착 결과는 바이트 단위로 같았다. 이 측정은 docs/geometry-pipeline.md 의 Exact-surface flight overhead after far-pair rejection (2026-10-25) 절에 있다. 측정은 다른 작업과 기계를 나눠 쓰는 동안 했다.

4.7 GPU 비행

이온은 서로 독립이므로 GPU 에서 이온 하나를 스레드 하나로 날린다. 정밀 모드는 배정밀도로 CPU 와 같은 적분기와 재구성을 쓰고 앱의 실시간 미리보기는 단정밀도 미리보기 모드를 쓴다. einzel 렌즈 10 만 개를 배정밀도 GPU 로 날리면 계산에 0.637 s 가 걸렸다. SIMION 기본 품질의 21.37 s 보다 33.5 배 짧다. 이 값은 bench/fly-gpu.md 의 Native precision study 절과 bench/simion-baseline.md 의 Fly 절에서 가져왔다.

GPU 커널에서는 전기장 재구성의 다항식 계산이 대부분의 시간을 쓴다. 다항식의 6 개 계수 반복을 풀어 쓰니 GPU 컴파일러가 더 좋은 코드를 만들었다. 그림 7 은 그 전후의 계산 시간이다. 가로 묶음은 템플릿과 이온 수, 회색과 파랑 막대는 개선 전후, 왼쪽은 미리보기 모드(f32), 오른쪽은 정밀 모드(f64)다. 미리보기 모드에서 4 중극 10 만 개가 99.7 ms 에서 54.9 ms 로 줄었고 결과는 개선 전과 비트 단위로 같다.

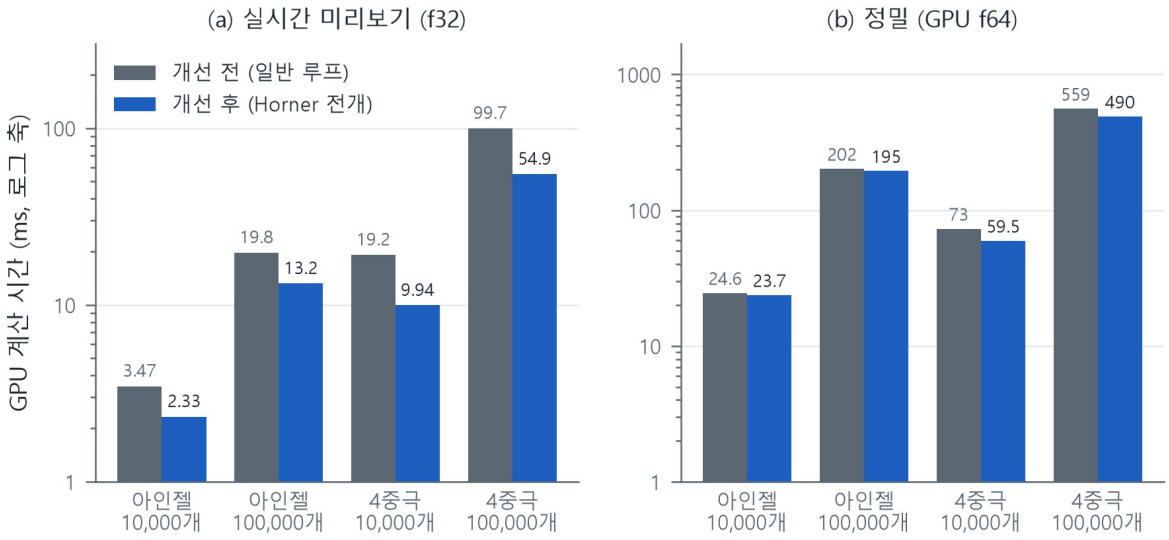


그림 7. GPU 비행 커널의 다항식 전개 전후 계산 시간(RTX 3070, Vulkan, 세 번 측정의 중앙값, 계산 패스 시간만). bench/fly-gpu.md 의 Cached-power Horner specialization 절 표를 이 문서에서 다시 그렸다. 출처: figures/F10.png.

CPU 와 GPU 가운데 무엇을 쓸지는 --device auto 가 측정값으로 정한다. 전기장만 있는 비행에서 배정밀도를 지원하는 GPU 는 이온 4,096 개부터 CPU 보다 빨랐다. 1,024 개에서는 CPU 13.97 ms, GPU 43.72 ms 였고 4,096 개에서는 CPU 58.73 ms, GPU 51.28 ms 였다. 이 값은 bench/fly-gpu.md 의 Automatic device selection 절에서 가져왔다. GPU 가 없거나 배정밀도를 지원하지 않으면 CPU 를 쓴다.

4.8 앱의 실시간 재비행

앱에서는 전압 슬라이더를 끄는 동안 이온을 계속 다시 날려 결과를 바로 보여 준다. 그림 8 은 4 중극 템플릿에서 슬라이더를 움직이는 실제 화면이다. 표 7 은 슬라이더를 5 초 동안 끌었을 때 초당 갱신 횟수다.

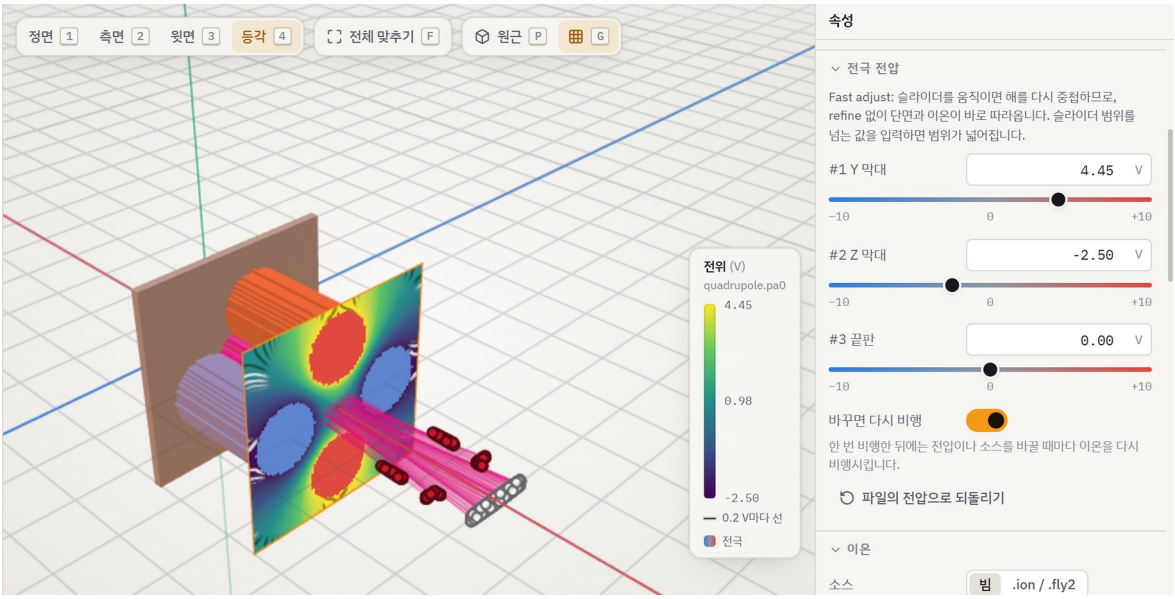


그림 8. 4 중극 템플릿에서 전극 전압 슬라이더를 움직이며 GPU 로 실시간 재비행하는 앱 화면. 출처: figures/F09.png.

템플릿, 이온 수	개선 전 (회/s)	현재 (회/s)	왕복 시간 (ms)
einzel 1 만	39.1	50.3	28
einzel 10 만	13.5	17.7	144
4 중극 1 만	30.9	39.5	66
4 중극 10 만	7.6	8.6	309

표 7 은 docs/dev/frontend.md 의 제출/수거 파이프라인과 작은 비행 패킷 절에 기록된 앱 측정값이다. 개선은 두 가지다. 다음 요청을 앞 요청의 GPU 계산과 겹쳐 제출했고 결과 패킷을 줄였다. 이 측정은 4.7 절의 커널 개선 전에 했으므로 4 중극 10 만 개의 현재 앱 갱신 횟수는 다시 재야 한다. 측정 당시 다른 프로그램이 CPU 코어 일부를 쓰고 있었다는 점도 기록에 남아 있다.

> **요점 정리**
>
> SimRep 은 식 (4)를 RK45(선택 시 Boris)로 적분하고, 칸 경계에서도 연속인 전기장으로 스텝을 크게 잡는다. einzel 렌즈에서 SIMION 보다 오차가 4.9 배 작으면서 11.5 배 빠르고(표 5), 3D 4 중극의 정확한 표면 경로는 오차를 약 700 배 줄이면서 71 배 빠르다(표 6). 전기장만 있는 비행에서 배정밀도를 지원하는 GPU 는 이온 4,096 개부터 자동으로 쓰인다(bench/fly-gpu.md 의 Automatic device selection 절).

5. 물리 확장

> **이 장에서 배울 것**
>
> 자기장, 중성 기체 충돌, 유전체, 공간전하, 재료가 있는 자기장 계산을 어떻게 넣었는지와 각각을 SIMION 과 같은 정밀도에서 비교한 결과를 정리한다.

SIMION 은 정전기 비행 외에도 자기장, 기체 충돌, 유전체, 공간전하를 다룬다. 이 장에서 다루는 기능은 모두 같은 원칙으로 검증했다. 먼저 해석해나 독립 기준해로 우리 계산의 수렴을 보이고 같은 기준해에 SIMION 결과를 대어 정밀도를 맞춘 뒤 계산 시간을 비교한다.

5.1 자기장

자기장은 세 가지로 준다. 균일한 자기장, 자극 형상을 풀어 얻은 자기 스칼라 포텐셜 배열(자기 PA), 전류가 흐르는 코일이다. 자기 PA 와 코일의 자기장은 식 (6)으로 계산한다.

$$\boldsymbol{B} = -n_g \nabla p, \boldsymbol{B}(\boldsymbol{x}) = \frac{\mu_0}{4\pi} \int \frac{I\, d\boldsymbol{\ell} \times \boldsymbol{r}}{|\boldsymbol{r}|^3} \quad (6)$$

식 (6)의 앞식에서 p 는 자기 PA 에 저장된 스칼라 포텐셜이고 ng 는 PA 머리말의 배율이다. 둘을 곱해 기울기를 취하면 자기장 B(G)가 된다. 뒷식은 비오-사바르 법칙으로 전류 I(A)가 흐르는 도선 조각 $d\boldsymbol{\ell}$ 이 거리 r 만큼 떨어진 점에 만드는 자기장을 더한다. 이 문서의 단위계(G, mm, A)에서는 $\mu_0/4\pi$ 가 1 G·mm/A 다. 원형 코일은 타원 적분으로 정확히 계산한다. 두 식과 단위 규약은 docs/physics-magnetic-collision.md 의 Units and magnetic fields 절에 있고 구현은 crates/simrep-fly/src/physics/magnetic.rs 의 Coil::field_gauss 함수다. SIMION 의 자기 포텐셜 정의는 SIMION 2024 도움말의 자기 포텐셜 문서(Scientific Instrument Services, 2024a)를 따랐다.

솔레노이드에 코일이 100 개 있으면 이온이 한 스텝 갈 때마다 코일 100 개의 타원 적분을 더해야 하므로 느렸다. 그래서 비행 전에 한 번, 솔레노이드의 축대칭을 이용해 (축 방향, 반지름) 격자 위에 자기장을 미리 계산해 두는 코일 지도를 만들었다. 지도는 간격 0.0625 mm 이고 반지름 성분은 축 근처에서도 정확하도록 B_r/r 형태로 저장한다. 지도를 쓰기 전에 영역마다 격자점 사이 512 곳에서 정확한 계산과 비교한다. 최대 차이가 2e-9 G 이하일 때만 지도를 쓴다. 코일 자기장은 이온이 바꾸지 않는 외부 장이므로 지도로 바꿔도 물리는 같다. 차이는 보간 오차뿐이다. 이 내용은 bench/magnetic-cpu.md 의 Implementation and profile 절에 있다. 그림 9 는 그 검사 결과다. 왼쪽은 이온 100 개를 지도와 정확한 계산으로 각각 날렸을 때의 도착 위치 차이(로그축)다. 모두 4e-7 mm 이하다. 계산 시간은 그림을 만들 때 다시 실행한 한 번의 측정에서 0.836 s 에서 0.243 s 로 줄었다. 오른쪽은 지도의 세 영역에서 표본 512 개로 잰 최대 오차다. 모두 허용선(2e-9 G, 상대 5e-10) 아래에 있다.

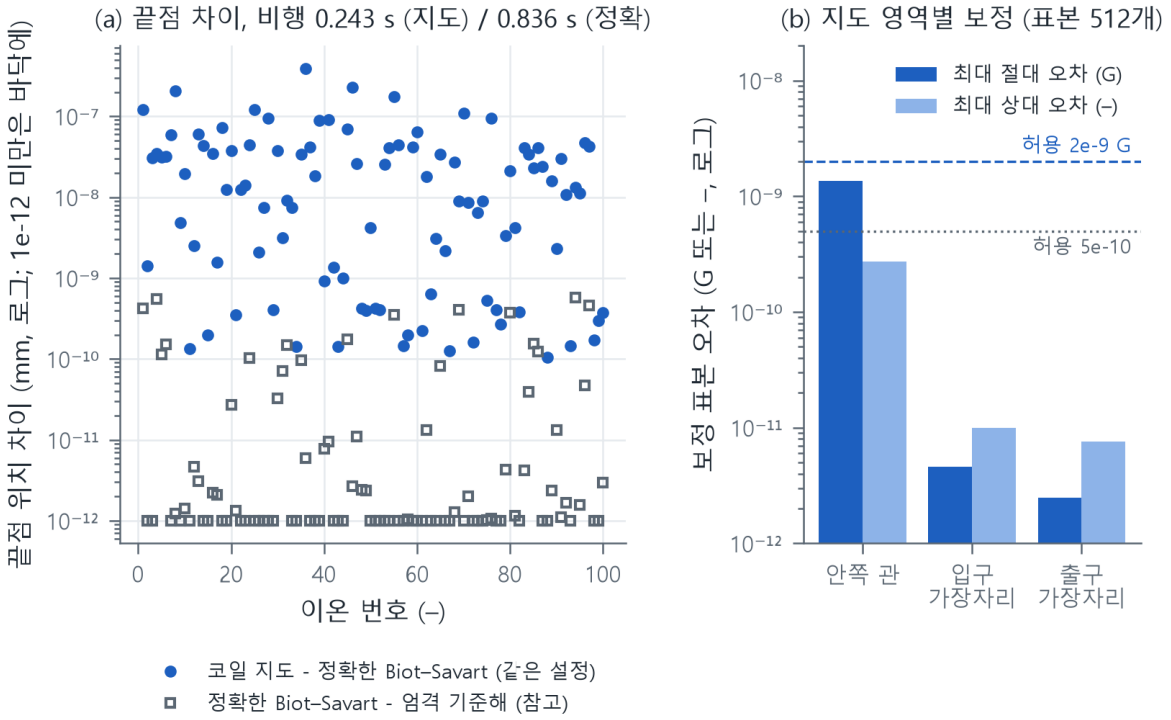


그림 9. 솔레노이드 코일 지도의 검증: 이온별 도착 위치 차이(왼쪽)와 영역별 보정 오차(오른쪽). 현재 빌드에서 실행한 검사 결과를 그렸다. 실행 기록은 figure-ledger.json 의 F12. 출처: figures/F12.png.

SIMION 자기장 예제 7 개를 같은 이온 출발 조건으로 날려 비교했다. 기준해는 정확한 비오-사바르 계산이나 원래 자기 PA 로 스텝 0.0025 μs, 허용 오차 1e-12 로 날려 얻었다. 일곱 경우 모두 도착 위치와 비행시간이 기존 정확도 기준 안에 들었다. 표 8 은 계산 시간이다.

예제	이온 수	SIMION (s)	SimRep CPU (s)	배수
magnet1	1,000	0.35	0.166	2.1
mag90	1,000	0.25	0.178	1.4
자기 섹터 2dp	150	0.24	0.033	7.3
자기 섹터 2dc	500	0.23	0.025	9.3
자기 섹터 3dp	150	0.17	0.063	2.7
헬름홀츠 코일	270	0.60	0.012	48.7
솔레노이드 (코일 지도)	100	4.77	0.222	21.5

표 8 은 bench/magnetic-cpu.md 의 Actual matched-population compute 절에서 가져왔다. SIMION 은 세 번, SimRep 은 다섯 번 측정한 중앙값이다. SimRep 은 CPU 16 코어다. 일곱 예제 모두 SimRep 이 빠르다. 그림 10 은 자기 섹터 예제를 실제로 날린 결과다. 왼쪽 궤적에서는 이온이 자기장 영역에서 원호를 그리며 약 90 도 휘어 세 곳의 끝점에 닿고 SIMION 의 끝점(

네모)과 겹친다. 오른쪽은 이온 15 개의 SIMION 대비 차이다. 위치는 0.01 mm 이하, 비행시간은 0.0003 μs 이하다.

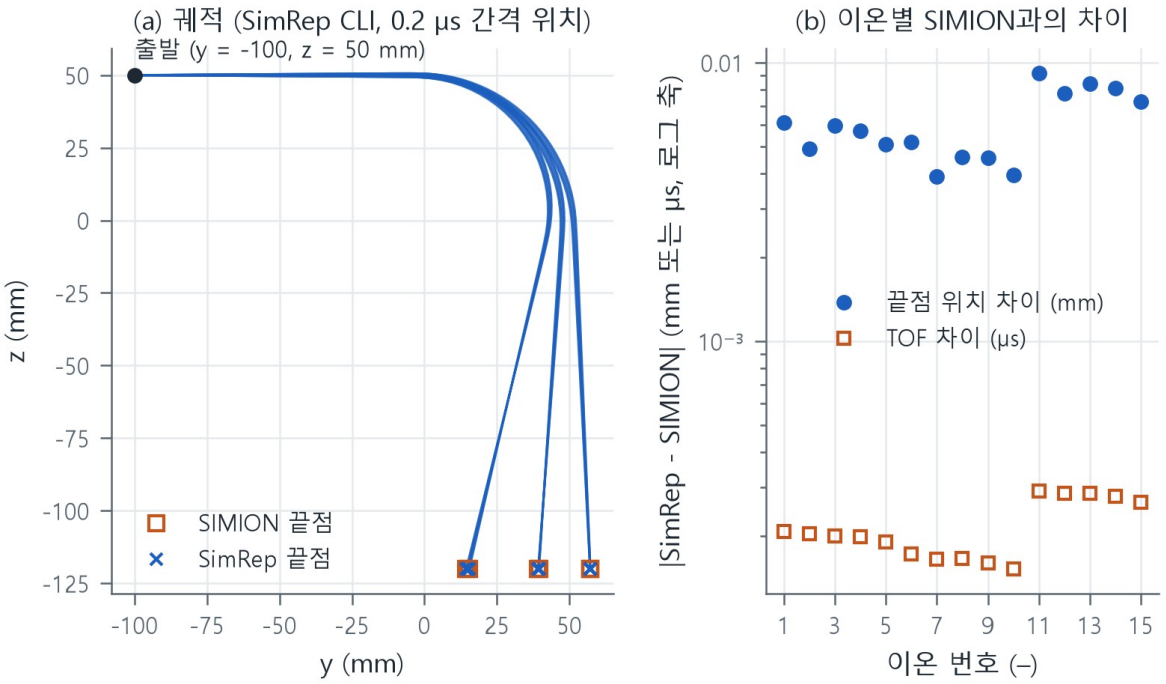


그림 10. 자기 섹터 예제의 이온 궤적(SimRep CLI 실행 결과). 실행 명령은 figure-ledger.json 의 F11. 출처: figures/F11.png.

자기장 계산은 GPU 보다 CPU 가 빨랐다. RTX 3070 의 배정밀도 연산은 단정밀도의 1/64 로 느리고 자기 PA 보간과 코일 계산은 배정밀도가 필요하기 때문이다. 자기 섹터 3dp 1 만 개를 계산할 때 GPU 는 4.03 s, CPU 는 1.14 s 였다(bench/fly-gpu.md 의 Magnetic fields and neutral gas 절). 그래서 --device auto 는 자기장 비행에 CPU 를 고른다. GPU 자기장 비행 자체는 구현되어 있어 --device gpu 로 실행할 수 있다. 배정밀도가 빠른 GPU 라면 결과가 달라질 수 있다.

5.2 중성 기체 충돌

기체 속 이온은 기체 분자와 부딪혀 방향과 에너지가 바뀐다. SIMION 예제의 두 모델을 구현했다. HS1 은 단단한 공 충돌 모델이다. 이온과 기체 분자의 상대 속도로 충돌 빈도를 정하고 무작위로 충돌을 일으킨다. 한 스텝 Δt 동안의 충돌 확률은 식 (7)이다.

$$n=\frac{P}{k_B T},\nu=n\sigma\left\langle\left|\boldsymbol{v}-\boldsymbol{v}_g\right|\right\rangle,P_{coll}=1-e^{-\nu\Delta t}\quad(7)$$

식 (7)의 n 은 압력 P(Pa)와 온도 T(K)로 정한 기체 수밀도(m⁻³)이고 k_B 는 볼츠만 상수(J/K)다. σ 는 충돌 단면적(m²)이고 ⟨|v − vg|⟩는 이온과 기체 분자의 평균 상대 속력이다. 식 (7)은 속력 m/s 와 시간 s 를 쓰는 SI 식이다. 앞의 mm/μs 속력을 넣으면 ν(μs⁻¹) = 10⁻³ n σ ⟨|v − vg|⟩가 된다. 셋을 곱한 ν 가 단위 시간당 충돌 횟수다. Δt 동안 한 번 이상 충돌할 확률이 Pcoll 이다. 기체가

짙거나 이온이 빠를수록 ν 가 커져 자주 부딪힌다. 이 식은 SIMION 에 들어 있는 examples/collision_hs1/collision_hs1.lua(Scientific Instrument Services, 2024c)의 규칙을 이 문서의 기호로 옮긴 것이다. 정리는 docs/physics-magnetic-collision.md 의 HS1 절, 구현은 crates/simrep-fly/src/physics/collision.rs 의 collision_rate_per_us 함수다. SDS 는 대기압처럼 충돌이 아주 잦은 경우를 위한 통계적 확산 모델로 SIMION 의 mbmr.dat 표를 그대로 쓴다.

무작위 모델은 이온 하나하나가 아니라 분포로 비교했다. SIMION 과 SimRep 으로 각각 이온 4,096 개를 날리고 도착 위치와 에너지 분포가 같은지 콜모고로프-스미르노프(KS) 검정으로 확인했다. 그림 11 은 HS1 에서 50 μs 뒤 x 위치와 운동 에너지의 누적 분포를 SIMION, SimRep CPU, SimRep GPU 로 겹쳐 그린 그림이다. 세 곡선이 거의 하나로 겹친다. SIMION 과의 KS 검정 p 값은 위치 성분에서 0.38~0.98, 에너지에서 0.38 이다. 5% 기준에서 같은 분포라는 가설을 기각하지 않았고 CPU 와 GPU 사이도 마찬가지다. SDS 도 p 값 0.36~1 로 같았다. 이 값은 docs/physics-magnetic-collision.md 의 Statistical results 절에 기록되어 있다. 현재 기본 설정의 계산 시간은 HS1 이 SimRep 0.183 s 대 SIMION 9.96 s(54 배), SDS 가 0.0049 s 대 1.23 s(251 배)였다(bench/default-flight.md 의 HS1 and SDS statistical checks 절).

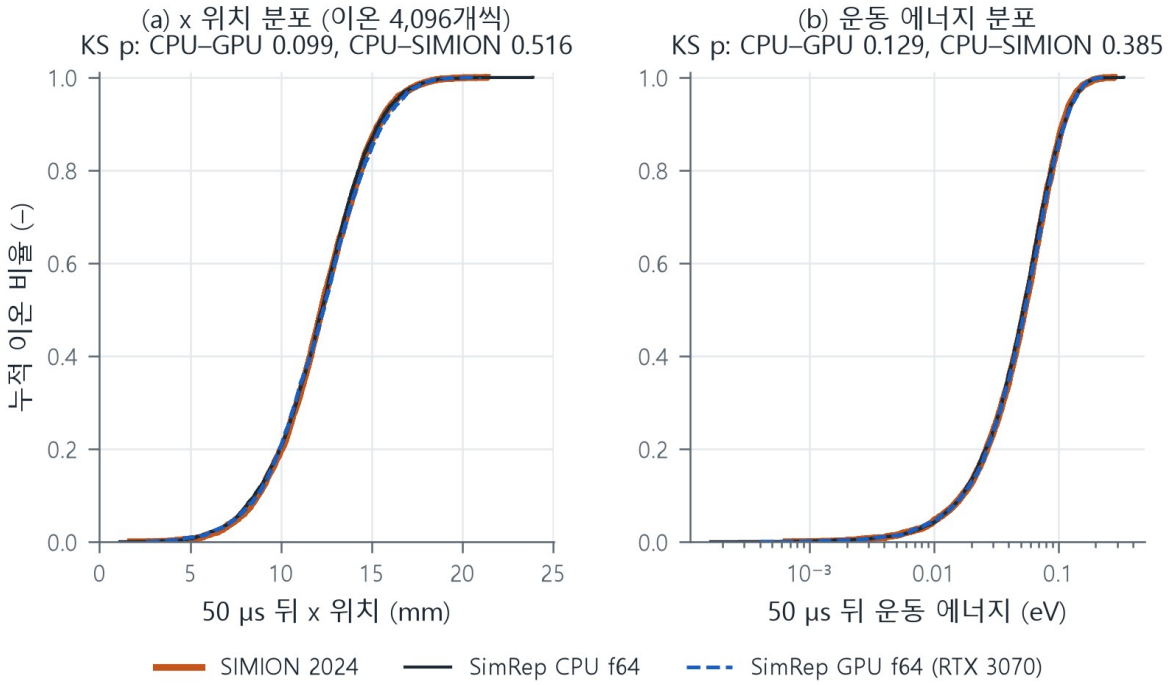


그림 11. HS1 충돌 모델에서 이온 4,096 개의 50 μs 뒤 위치·에너지 누적 분포(SIMION, SimRep CPU, SimRep GPU). 출처: figures/F13.png, 자료와 명령은 figure-ledger.json 의 F13.

5.3 유전체

유전체는 전기장 안에서 분극되어 전위 분포를 바꾸는 절연체다. 유전율이 위치마다 다르면 식 (1) 대신 식 (8)을 푼다.

$$-\nabla\cdot(\varepsilon_r\nabla V)=\frac{\rho_f}{\varepsilon_0},\varepsilon_{ij}=\frac{2}{1/\varepsilon_i+1/\varepsilon_j}\quad(8)$$

식 (8)의 ε_r 은 상대 유전율, ρf 는 자유 전하 밀도이며 ρf 와 ε_0 의 단위는 식 (1)과 같다. 앞식은 전기 변위의 발산이 자유 전하와 같다는 가우스 법칙이다. ε_r 이 모든 곳에서 1 이면 식 (1)로 돌아간다. 뒷식은 격자점마다 유전율을 준 입력에서 이웃한 두 점의 유전율로 둘을 잇는 계수를 정하는 조화평균이다. SIMION 유전율 PA 처럼 칸마다 값을 준 입력에서는 연결을 가로지르는 칸들의 면적 산술평균을 쓴다(from_cells 함수). 조화평균을 쓰면 유전체 경계에서 전기 변위의 수직 성분이 연속이라는 조건이 자연스럽게 지켜진다. 행렬도 대칭으로 남아 3 장의 멀티그리드를 그대로 쓴다. 이 이산화는 docs/dielectric.md 의 Discretization and interfaces 절에 있고 구현은 crates/simrep-solve/src/dielectric.rs 의 harmonic 함수다. SIMION 의 유전체 정의는 SIMION 2024 도움말의 유전체 문서(Scientific Instrument Services, 2024b)를 따랐다.

SIMION 유전체 예제 세 개를 SimRep 은 잔차 1e-10 으로 풀고 SIMION 은 엄격 설정(반복 사이 전압 변화 1e-7 V)으로 풀어 비교했다. 표 9 는 자동으로 고른 장치의 재료 준비, 계층 구성, 풀이 시간을 합한 값이다.

예제	자동 선택 장치	SimRep (s)	SIMION 엄격 설정 (s)	배수
평행판 plates_2dp	CPU	0.065	0.13	2.0
구 sphere_2dc	CPU	0.184	0.43	2.3
구 sphere_3dp	GPU	1.101	25.78	23.4

표 9 는 docs/dielectric.md 의 Automatic selection at the default residual tolerance 절에 있는 값을 합한 것이다. 이 측정은 다른 작업과 기계를 나눠 쓰는 동안 했다. 3D 구에서는 GPU 데이터 전송까지 넣어도 1.41 s 로 SIMION 보다 18 배 빠르다. 장치는 측정한 교차점으로 고른다. 3D 격자에서 선형 유전체는 1,048,576 점 이상, 비선형 유전체는 131,072 점 이상이면 GPU 를 쓴다 (docs/dielectric.md 의 Class-aware auto thresholds 절). 그림 12 는 sphere_3dp 의 결과다. 왼쪽 전위 단면에서 유전율 10 인 구 안의 등전위선 간격이 넓어지는데, 이는 구가 분극되어 내부 전기장이 약해졌다는 뜻이다. 오른쪽은 축 위의 전기장으로, 바깥에서 걸어 준 1 V/mm 가 구 표면 바깥에서 커지고 구 안에서 0.2572 V/mm 로 줄어든다. SIMION 과 SimRep 의 두 선이 겹친다.

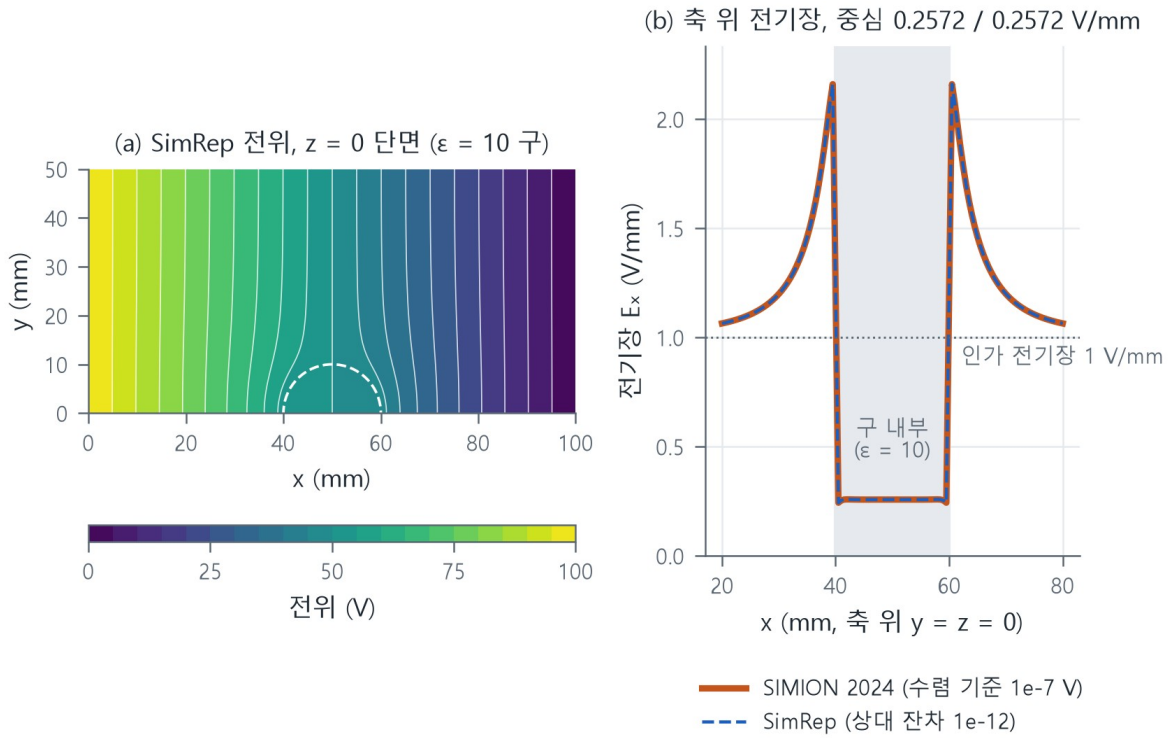


그림 12. 유전체 구 예제의 전위 단면(SimRep)과 축 위 전기장(SimRep 과 SIMION 엄격 설정). 출처: figures/F14.png, 명령은 figure-ledger.json 의 F14.

유전율이 전기장 세기에 따라 바뀌는 비선형 유전체도 넣었다. 전위를 풀고 그 전기장으로 유전율을 다시 계산하는 반복을 수렴할 때까지 되풀이한다. SIMION 의 비선형 구 예제에서는 처음에 SIMION 과 최대 425 V 차이가 났다. 원인을 찾아보니 두 프로그램이 유전율을 정할 전기장을 서로 다르게 계산했다. SIMION 은 반 칸 떨어진 전위 차이를 쓰고 SimRep 은 칸 안에서 전위를 미분한다. 격자 간격을 2, 1, 0.5, 0.25 mm 로 줄여 보니 이 차이는 1081 V, 625 V, 425 V, 226 V 로 줄었다. 두 방식이 같은 연속 극한으로 향한다고 판단했다(docs/dielectric.md 의 Physical grid-refinement audit 절). 그래서 연속 극한 추정값을 기준으로 비교했다. 일곱 관측량(탐침 전위 네 개, 중심 전기장, 유도 쌍극자, 판 전하)의 추정 오차가 모두 SIMION 기본 설정 이하인 가장 싼 시험 설정은 SimRep 2 배 격자(간격 0.25 mm)였고(같은 문서의 Speed at matched estimated physical accuracy 절), 이때 CPU 43.6 s, GPU 48.4 s 로 SIMION 87.5 s 보다 각각 2.0 배, 1.8 배 빨랐다.

5.4 공간전하

이온이 많으면 이온끼리의 쿨롱 반발이 궤적을 바꾼다. SimRep 은 모든 이온을 같은 시각에 맞춰 함께 움직이며 서로의 전기장을 계산한다. 가장 직접적인 방법은 식 (9)의 쌍별 합이다.

$$\boldsymbol{E}_i = k \sum_{j \neq i} \frac{Q_j \boldsymbol{r}_{ij}}{\left(|\boldsymbol{r}_{ij}|^2 + \varepsilon^2 \right)^{3/2}}, \quad k = \frac{1}{4\pi\varepsilon_0} \tag{9}$$

식 (9)의 E_i 는 i 번째 이온 위치의 공간전하 전기장(V/mm), Q_j 는 j 번째 이온이 나타내는 전하(C), r_{ij} 는 j 에서 i 로 향하는 벡터(mm)다. ε 는 아주 가까운 두 이온의 힘이 무한대가 되지 않게 하는 완화 길이다. 0 이면 정확한 점전하 쿨롱 힘이다. 이 합은 이온 수의 제곱에 비례해 늘어난다. 이온이 많아지면 전하를 격자에 나눠 놓고 식 (1)의 푸아송 방정식을 푸는 PIC 방식을 쓴다. 두 방식은 docs/physics-space-charge.md 의 Direct Coulomb 절과 PIC and Poisson 절에 있다. 쿨롱 법칙을 이 문서의 단위로 옮긴 식이다.

SIMION 예제의 이온 103 개로 비교했을 때 SimRep 의 도착 위치 오차는 독립 기준해 대비 1.0e-4 mm 였고 SIMION 은 0.59 mm 였다. 계산 시간은 비교용으로 보존한 one-cell 설정에서 0.249 s 대 0.93 s 였다(bench/default-flight.md 의 Retained one-cell baseline 절). SIMION 의 beam 모드는 이온 하나가 전류 한 줄기를 나타내는 방식이다. SIMION 문서에 적힌 대로 첫 이온이 정하는 평면에 맞춰 나머지 이온을 함께 옮기도록 구현했다. 그림 13 은 균일한 원통 빔이 자기 반발로 퍼지는 반경을 해석해와 겹쳐 그린 것으로, 두 선이 겹친다. SIMION beam 예제 두 개에서 SimRep 은 더 작은 오차로 각각 3.19 배(0.269 s 대 0.86 s), 1.99 배(2.117 s 대 4.21 s) 빨랐다(bench/space-charge.md 의 Matched-precision compute verdict and defaults 절). 이온 10 만 개의 PIC 는 전역 스텝 3 회의 짧은 검사에서 GPU 가 CPU 보다 1.4 배 빨랐고 두 결과의 차이는 3e-16 mm 였다(bench/space-charge.md 의 GPU PIC agreement and compute time 절).

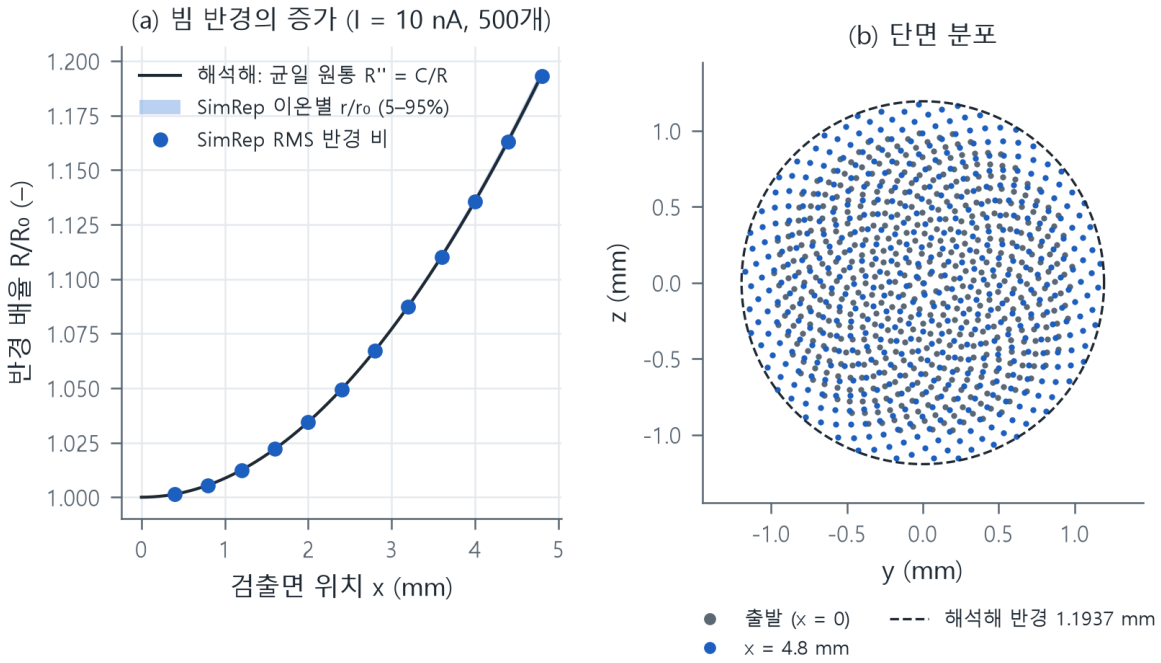


그림 13. 균일한 원통 빔의 공간전하 퍼짐: SimRep beam 모드 결과와 해석해. 출처: figures/F15.png, 명령은 figure-ledger.json 의 F15.

5.5 재료가 있는 자기장

철심, 영구자석, 전류 도체가 있으면 자기장도 방정식을 풀어야 한다. SIMION 의 자기 포텐셜 예제를 따라 두 가지 꼴을 넣었다. 철심과 자석은 식 (10)의 스칼라 포텐셜로 푼다.

$$-\nabla\cdot(\mu_r\nabla p)=\frac{4\pi}{n_g}(-\nabla\cdot\boldsymbol{M}),\boldsymbol{B}=-n_g\mu_r\nabla p+4\pi\boldsymbol{M}\quad(10)$$

식 (10)의 μr 은 상대 투자율, M 은 자석의 자화(A/mm)다. 앞식은 자화가 만드는 자극을 원천으로 하는 포텐셜 방정식이고, 형태가 식 (8)과 같아서 유전체 솔버의 계수 계산을 그대로 쓴다. 뒷식은 그 포텐셜에서 자기장을 얻는 식이다. 이온이 날아가는 진공에서는 μr = 1, M = 0 이므로 뒷식이 식 (6)의 앞식으로 돌아가고, 결과를 기존 자기 PA 형식으로 저장해 그대로 날릴 수 있다. 이 식과 비행 규약은 docs/magnetostatics.md 의 Equations and flight contract 절에 있다. 전류는 식 (11)의 벡터 포텐셜로 푼다.

$$-\nabla\cdot\left(\mu_r^{-1}\nabla A_z\right)=\frac{4\pi}{n_g}J_z,\boldsymbol{B}=n_g\nabla\times\boldsymbol{A}\quad(11)$$

식 (11)의 Jz 는 평면에 수직으로 흐르는 전류 밀도(A/mm²), Az 는 그 방향의 벡터 포텐셜이며 B = ng ∇ × A 에 맞춰 G·mm/ng 단위로 저장한다. 자기장은 벡터 포텐셜의 회전(curl)으로 얻는다. 축대칭 문제는 ψ = rAφ 를 평면 발산 형태의 식으로 푼다. 이는 원통 좌표의 스칼라 라플라시안과 다르다. 3D 문제에서 세 성분을 각각 푸는 것은 μr 이 공간적으로 일정할 때만 성립한다. 식 (10)과 (11)은 SIMION 2024 도움말의 자기 포텐셜 문서(Scientific Instrument Services, 2024a)의 정의를 이 문서의 기호로 옮긴 것이다. 정리는 docs/magnetostatics.md 의 Equations and flight contract 절, 구현은 crates/simrep-solve/src/magnetostatics.rs 다.

SIMION 자기 포텐셜 예제 20 개, 계산 25 건으로 비교했다. 25 건 모두 저장된 포텐셜 기준으로 우리 결과가 SIMION 기본 설정보다 기준해에 가까웠고 계산 시간은 22 건 가운데 21 건에서 1.7~23 배 빨랐다(docs/magnetostatics.md 의 Audit results 절). 처음에 느렸던 균일장 예제는 내부를 0 에서 시작해 필요 없는 반복을 27 번 했기 때문이었다. 경계가 평면이고 재료가 없으면 정확한 선형 해에서 시작하도록 고쳤다. 그 결과 0.704 ms 로 SIMION 의 24 ms 보다 34 배 빨라졌다(같은 문서의 Hardware GPU and uniform-field follow-up 절). 그래서 22 건 모두 SIMION 보다 빠르다. 그림 14 는 C 자 자석 예제의 자기장 크기 지도다. 자기장이 철심 안에 모이고 간극에는 균일한 자기장이 생긴다.

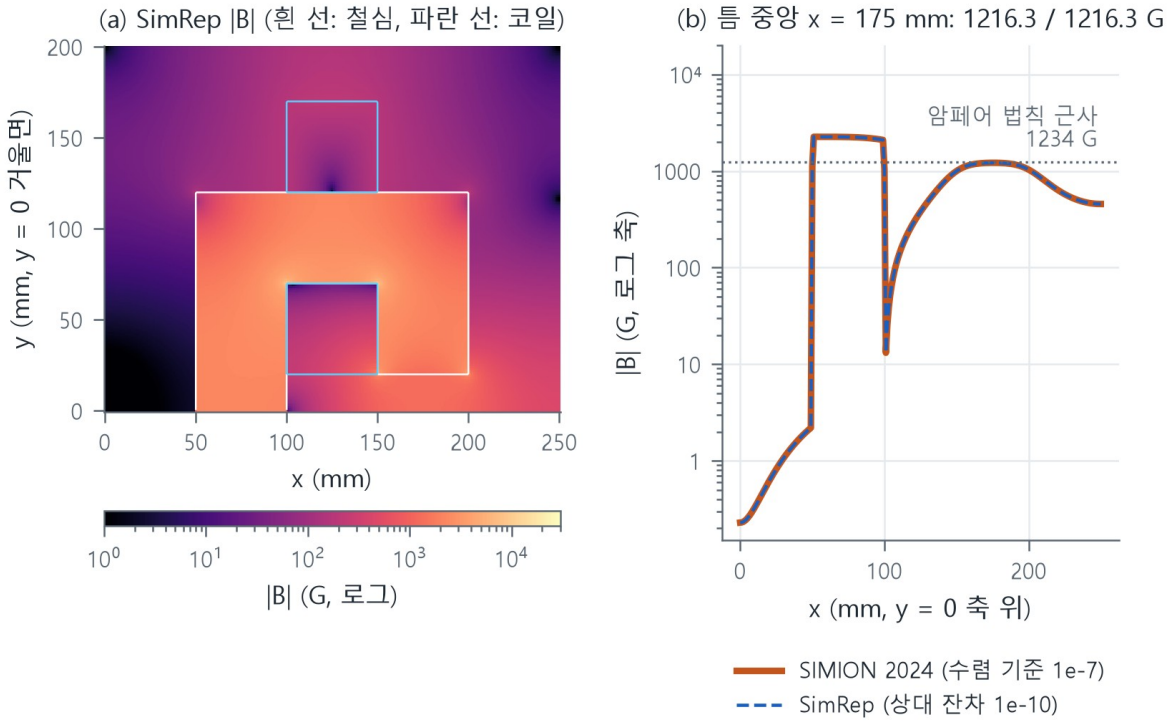


그림 14. C 자 자석 예제(c_magnet_2dp)의 자기장 크기 분포(SimRep 자기정역학 계산 결과). 출처: figures/F16.png, 명령은 figure-ledger.json 의 F16.

5.6 외부 프로그램의 자기장 지도

자석 설계에는 흔히 COMSOL, Opera, CST, FEMM 같은 전자기 해석 프로그램을 쓴다. 그 결과를 다시 계산하지 않고 바로 쓰도록 이 프로그램들이 내보낸 자기장 격자 파일을 읽어 비행에 쓰는 기능을 넣었다. COMSOL 의 Spreadsheet·Grid 형식, Opera 의 .table, 일반 CSV·텍스트, 축대칭 (z, r, Bz, B_r) 표를 읽고 길이 단위(mm, m, cm, inch)와 자기장 단위(G, T, mT), 위치·회전·배율 변환을 지정한다. 읽은 격자는 코일 지도와 같은 C2 보간으로 재구성하며 다른 자기장 소스와 겹쳐 더한다. 사용법은 docs/bfield-import.md 에 있다.

정확도는 정확한 해를 아는 자기장을 각 형식으로 내보낸 뒤 다시 읽어 날리는 방식으로 확인했다. 표 10 은 원형 코일과 솔레노이드를 3D 격자로 내보냈을 때 정확한 자기장으로 날린 결과와의 도착 위치·비행시간 차이다.

자기장	격자 간격 (mm)	도착 위치 차이 (mm)	비행시간 차이 (μs)
원형 코일	0.8	9.90e-7	7.94e-8
원형 코일	0.4	1.04e-7	8.29e-9
원형 코일	0.2	4.94e-9	4.28e-10
솔레노이드	0.8	5.08e-7	6.63e-8
솔레노이드	0.4	3.78e-8	4.84e-9

솔레노이드	0.2	1.86e-9	2.56e-10

표 10 은 bench/bfield-import.md 의 Cartesian full-flight errors 절에서 가져왔다. 격자 간격을 절반으로 줄일 때마다 차이가 약 10~20 배 줄어든다(관측 차수 3.2~4.4). 일반 CSV, Opera, COMSOL 두 형식으로 내보낸 같은 자기장은 모두 같은 오차를 냈다. 그림 15 는 이 수렴을 그림으로 보여 준다. 가로축은 격자 간격, 왼쪽 세로축은 도착 위치 차이(mm), 오른쪽 세로축은 비행시간 차이(μs)이고 둘 다 로그축이다. 실선은 3D 격자, 빈 표식의 점선은 축대칭 표다. 모든 곡선이 같은 차수로 줄지는 않는다. 섹터 자기장의 비행시간 차이는 이미 4e-11 μs 아래여서 0.4 mm 단계에서 오히려 커졌다. 지도를 읽고 재구성하는 데는 9,261 점 격자에서 약 25 ms 가 걸렸다. 다만 이 기능은 규칙 격자 파일만 받는다. 점이 불규칙하게 흩어진 데이터는 원래 프로그램에서 규칙 격자로 다시 내보내야 한다.

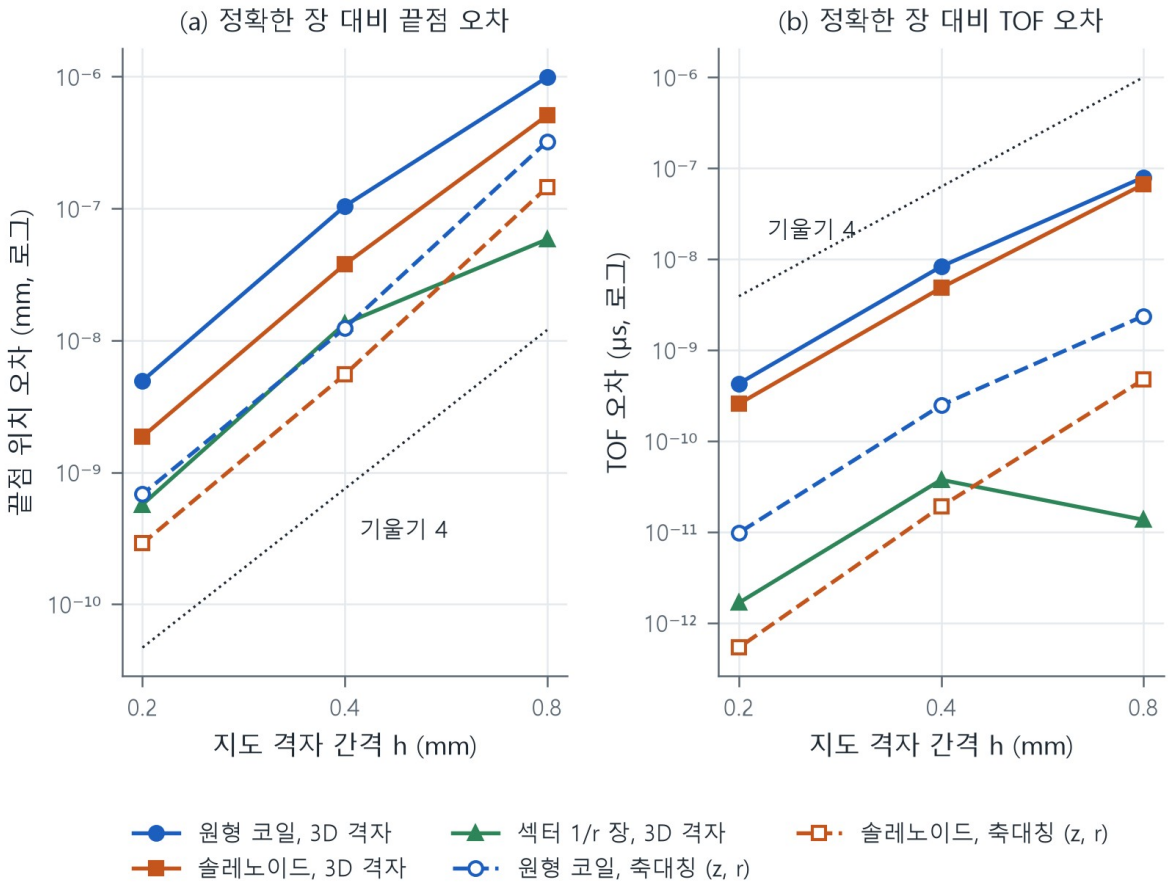


그림 15. 외부 자기장 지도의 격자 간격에 따른 도착 위치 차이(왼쪽)와 비행시간 차이(오른쪽)(정확한 자기장 대비). bench/bfield-import-20261024.csv 의 측정값을 이 문서에서 다시 그렸다. 출처: figures/F17.png.

> **요점 정리**
>
> 자기장, 기체 충돌, 유전체, 공간전하, 재료가 있는 자기장을 각 절의 기준으로 SIMION 예제와 비교했다. 외부 해석 프로그램의 자기장 지도도 읽어 쓸 수 있다. 자기장 예제 7 개(표 8), 유전체 예제 3 개(표 9), beam 예제 2 개(bench/space-charge.md 의 Matched-precision compute verdict and defaults 절), 자기 포텐셜 계산 22 건(docs/magnetostatics.md 의 Audit results 절과 Uniform Az root cause and fix 절)에서 모두 SimRep 이 빨랐다. 자기장 비행은 RTX 3070 에서 CPU 가 더 빨라 자동 선택이 CPU 다.

6. SIMION 파일과 3D 모델 호환

> **이 장에서 배울 것**
>
> SIMION 사용자가 가진 파일(PA, GEM, FLY2, ION, 작업대)과 3D 모델(STL, STEP, IGES)을 얼마나 그대로 쓸 수 있는지 정리한다.

새 프로그램을 쓰려면 기존 SIMION 작업 파일을 그대로 열 수 있어야 한다. 그래서 형식별로 정한 범위에서 SIMION 자신의 결과와 비교했다. GEM 261 개와 FLY2 154 개는 설치 예제 전체다. IOB 배치 148 개와 PA 정제·왕복 검사는 정해 둔 표본이다. 표 11 이 그 결과를 보여 준다.

형식	내용	SIMION 예제 비교 결과
PA, PA#, PA0	전위 배열, 전극 정의, fast adjust 기저	SIMION 이 SimRep 의 PA0 를 읽고 전압 조정까지 수행
GEM	구성 형상 정의와 GEM 안의 Lua 전처리	261 개 중 254 개 일치, 7 개는 단독 실행이 안 되는 include 조각, 미지원 0
FLY2	Lua 로 쓴 입자 출발 조건	154 개 중 97 개 순서까지 일치, 57 개 통계적 일치, 불일치 0
ION	입자 목록	지원
IOB	작업대(PA 배치, 작업대 범위)	앱이 설치 예제 148 개의 배치를 검증

표 11 의 GEM 결과는 docs/gem-validation.md 의 요약 절, FLY2 결과는 docs/fly2-validation.md 의 결과 표, PA 결과는 bench/refine-real.md 의 Validation and Windows process hygiene 절, IOB 는 docs/dev/frontend.md 의 SIMION 작업대 절에서 가져왔다. ION 목록은 docs/fly2.md 에 적힌 read_particles 함수로 읽는다. GEM 과 FLY2 파일은 사실상 Lua 프로그램이다. GEM 은 반복문과 함수로 전극을 그리고 FLY2 는 분포 함수로 입자를 만든다. 그래서 Lua 5.1.5 를 제한된

실행 환경으로 넣어 두 형식을 함께 쓰게 했다. 메모리 256 MiB 와 명령 50 억 개로 실행을 묶고 파일 쓰기 같은 외부 접근은 막는다(docs/gem.md). 무작위 분포를 쓰는 FLY2 는 SIMION 과 난수 순서가 달라 이온 하나하나를 비교할 수 없으므로 씨앗값 8 개로 KS 검정을 해 분포가 같은지 확인했다(docs/fly2-validation.md). 그림 16 은 Lua 반복문으로 전극을 그리는 hexapole.gem 을 1408×705 격자로 변환한 결과를 보여 준다. 확대한 오른쪽 그림에서 SimRep 의 전극 칸과 SIMION 의 전극점(검은 점)이 하나도 어긋나지 않는다.

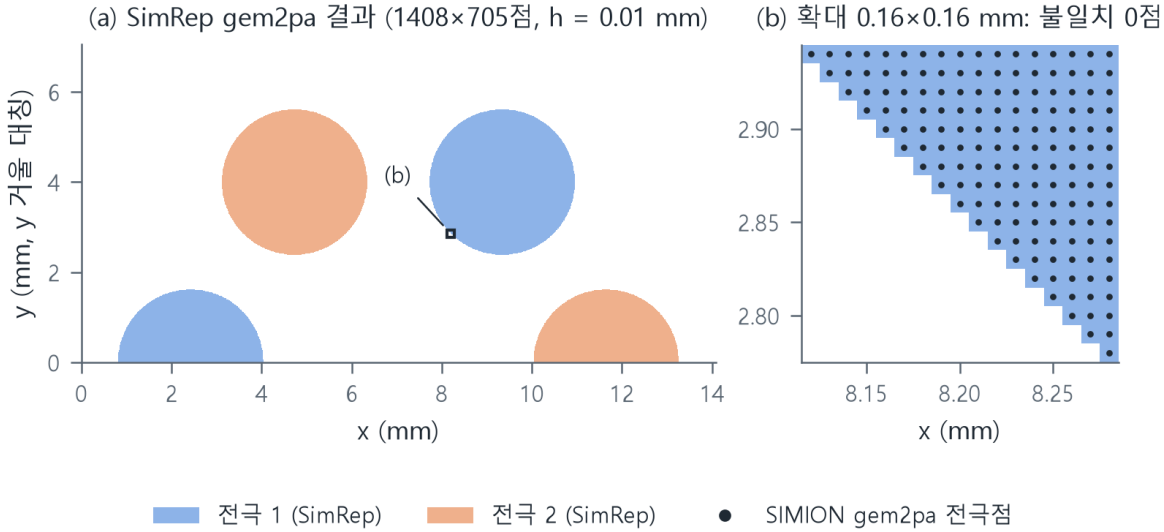


그림 16. Lua 반복문을 쓰는 GEM 파일(hexapole.gem)을 simrep gem2pa 로 변환한 전극 단면. 출처: figures/F19.png, 명령은 figure-ledger.json 의 F19.

FLY2 는 입자 목록 자체를 비교했다. 그림 17 의 위쪽은 출발 위치와 시각을 격자로 정한 결정적 정의(bngrid.fly2, 입자 231 개)로 SIMION 이 만든 목록(주황 원)과 SimRep 목록(파랑 점)이 모든 입자에서 겹치고 모든 열의 최대 차이가 1.1e-16 이다. 아래쪽은 무작위 정의 하나(tof_histogram.fly2)에서 뽑은 두 관측량을 비교한다. 가우스 분포로 뽑은 출발 시각(KS p = 0.60)과 원판에 고르게 뽑은 반지름 제곱(KS p = 0.75) 모두 SIMION 의 분포와 구별되지 않는다.

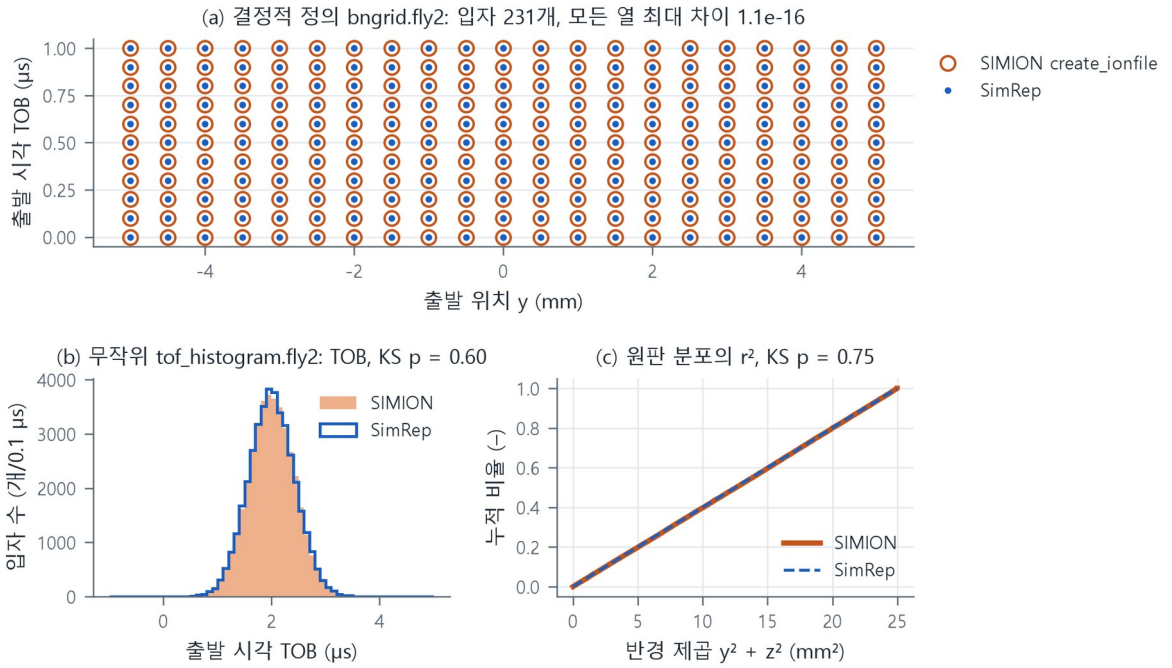


그림 17. FLY2 입자 출발 위치: SIMION 이 만든 목록과 SimRep 이 만든 목록. 출처: figures/F20.png, 자료는 figure-ledger.json 의 F20.

SIMION 도 GEM 파일에서 STL 을 가져온다. SimRep 은 여기에 OBJ, PLY, 3MF, glTF 메시와 STEP·IGES B-rep 가져오기를 더했다. 메시는 그대로 읽어 전극으로 쓴다(docs/mesh-io.md). STEP 과 IGES 는 OpenCascade 라이브러리로 읽는다(Open Cascade SAS, n.d.). CAD 모델은 정확한 B-rep 표면으로 거리를 계산하거나 삼각형으로 나눈 표면을 쓴다. 표 12 는 해석해가 있는 세 형상을 같은 격자에서 두 방식으로 풀었을 때의 전위 오차를 비교했다.

형상, 격자	정확한 표면 RMS/최대 (V)	삼각형 표면 RMS/최대 (V)
구, 33^3	0.00135 / 0.0128	0.00519 / 0.0323
원통, 65^2	0.00022 / 0.00080	0.00133 / 0.00376
고리, 25^3	0.00383 / 0.0404	0.00579 / 0.0321

표 12 는 docs/step-import.md 의 Measured accuracy and timing 절에서 가져왔다. 정확한 표면이 RMS 오차를 1.5~6 배 줄인다. 다만 고리의 최대 오차는 정확한 표면 쪽이 더 크다. B-rep 거리 계산의 준비 시간은 세 형상에서 0.46~6.38 s 로 삼각형 방식(0.0099~0.068 s)보다 훨씬 느리다. 그림 18 은 앱의 가져오기 마법사에서 두 구로 된 STEP 파일(pair.step)을 연 실제 화면이다. OpenCascade 가 고체 두 개를 읽어 둘 다 정상 B-rep 이라고 확인하고 사용자는 단위와 부품별 전극 번호·전압을 정한다. 다음 단계에서 격자 간격 추천을 받고 cut-cell 과 격자 정렬 방식 가운데 하나를 고른다.

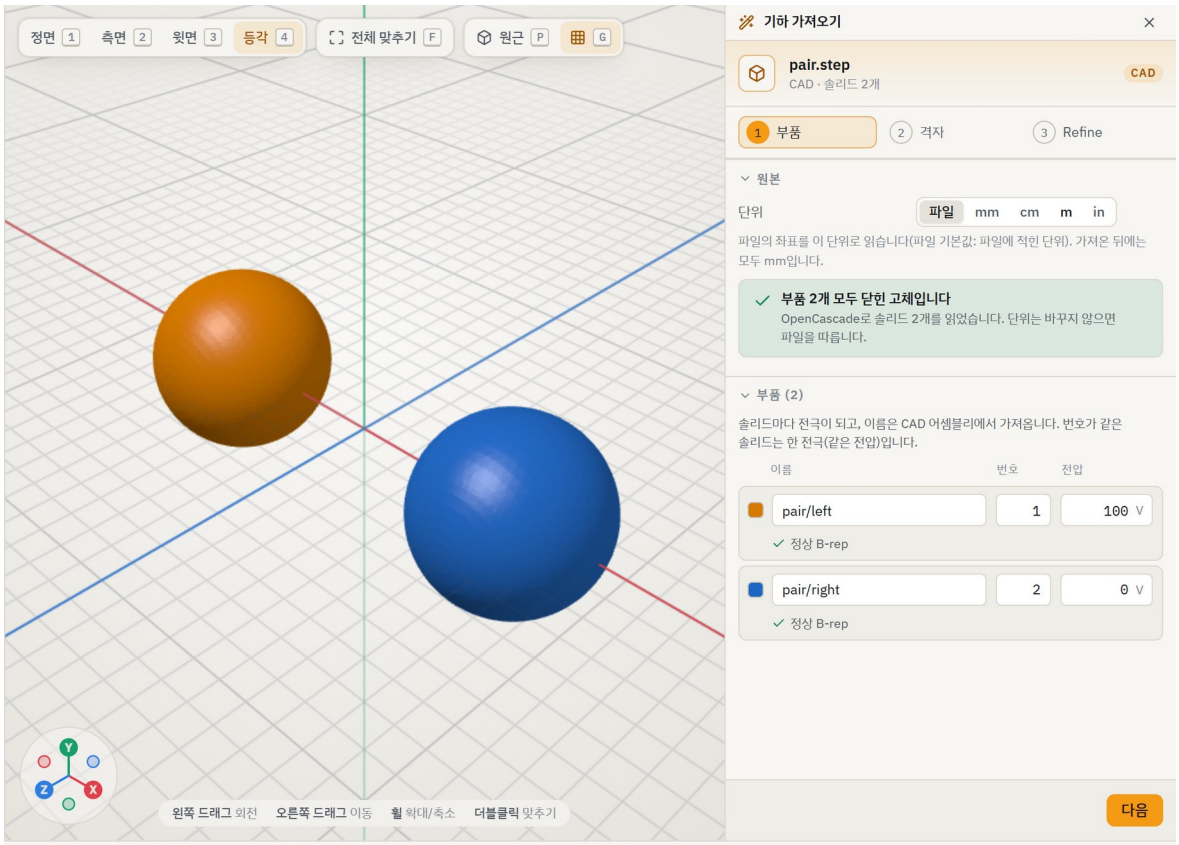


그림 18. 앱의 가져오기 마법사에서 STEP 파일(pair.step)을 여는 화면. 출처: figures/F18.png, 캡처 조건은 figure-ledger.json 의 F18.

내보내기는 전극 표면(STL, OBJ, glTF), 전위장(VTK 의 VTI·VTS), 궤적(CSV, VTP), 등전위면으로 한다(docs/mesh-io.md). 원래 계획에 있던 STEP 내보내기는 아직 넣지 않았다.

> **요점 정리**
>
> SIMION 예제의 GEM 261 개 가운데 단독 실행되는 254 개가 일치하고 나머지 7 개는 include 조각이다. FLY2 154 개는 97 개가 순서까지, 57 개가 통계적으로 일치한다(표 11). 메시와 STEP·IGES 모델을 전극으로 가져올 수 있으며, CAD 의 정확한 표면은 삼각형 표면보다 RMS 전위 오차가 작다(표 12).

7. 데스크톱 앱

> **이 장에서 배울 것**
>
> 사용자가 실제로 쓰는 데스크톱 앱의 흐름과 쉽게 쓰도록 넣은 기능을 정리한다.

앱은 Tauri 2 와 React 로 만들었고 3D 화면은 WebGPU 로 그린다. 계산은 2 장의 Rust 코어를 그대로 부른다(docs/dev/frontend.md 의 구조 요약 절). 시작 화면에서 einzel 렌즈, 정전기 4 중극, 반사형 TOF, 이온 트랩 템플릿을 고르면 형상 만들기, 전위 풀기, 전압과 입자 정하기, 날리기를 차례로 따라간다. 그림 19 는 4 중극 GEM 파일로 그 흐름을 실제로 따라간 화면 네 장이다. 첫째 장에서 가져오기 마법사가 GEM 의 부품 세 개를 보여 주고 전극 번호와 전압(2.5 V,

−2.5 V, 0 V)을 받는다. 둘째 장에서 cut-cell 격자와 보조 파일(PA#, cuts)을 만들고(135 ms, 표면이 지나는 점 11,160 개) 전위를 풀어 비행 준비를 마친다. 셋째 장에서 정확한 표면으로 이온 1,000 개를 날리고 넷째 장에서 TOF 히스토그램과 도착한 곳별 이온 수(+X 경계 750, 전극 3 에 230, 전극 2 에 20)를 본다. 이 개수는 4.6 절의 CLI 결과와 같다.

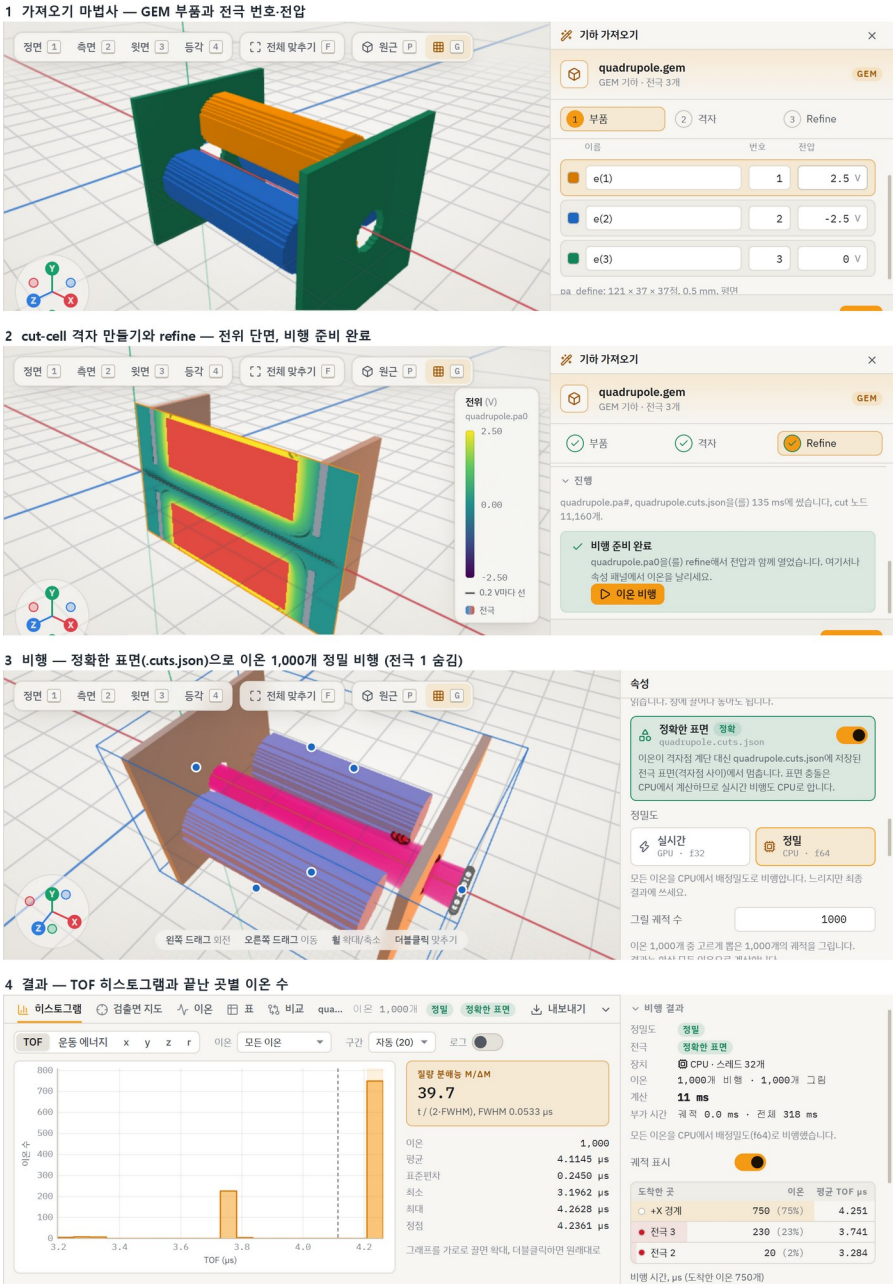


그림 19. 앱의 처리 흐름: 가져오기 → 전위 풀기 → 비행 → 결과를 연속으로 캡처했다. 출처: figures/F03.png, 캡처 조건은 figure-ledger.json 의 F03.

쉽게 쓰도록 다음을 넣었다. 처음 쓰는 사람을 위한 5 단계 따라하기가 있다. 실제 템플릿으로 전압 조정, 비행, TOF 히스토그램, CSV 내보내기까지 진행한다. 그림 20 은 그 둘째 단계로, 안내 상자가 집속 전극 슬라이더를 강조하고 전압을 바꾸면 무엇이 달라지는지 설명한다. 전압 슬라이더를 움직이면 결과를 실시간으로 다시 계산한다(4.8 절).



그림 20. 앱의 5 단계 따라하기 가운데 한 단계(한국어 화면). 출처: figures/F21.png.

정확한 표면 파일이 있으면 카드와 배지로 보여 주고 스위치로 끈다. 그림 21 은 같은 4 중극에서 스위치를 켠 경우(정밀 모드, CPU 배정밀도)와 끈 경우(실시간 미리보기, GPU 단정밀도)를 나란히 놓은 화면이다. 켜면 이온 20 개가 전극 2 의 실제 표면에 닿는다. 끄면 전극 2 에 닿는 이온이 없어져 도착 분류가 750/230/20 에서 751/249 로 바뀐다. 결과 패널은 어느 모드로 계산했는지 배지로 함께 보여 준다. 그 밖에 되돌리기와 다시 하기, 변경 후 1.5 초가 지나면 자동 저장, 한국어와 영어, 밝은 테마와 어두운 테마, 궤적과 결과 표의 연동 선택이 있다(docs/dev/frontend.md).

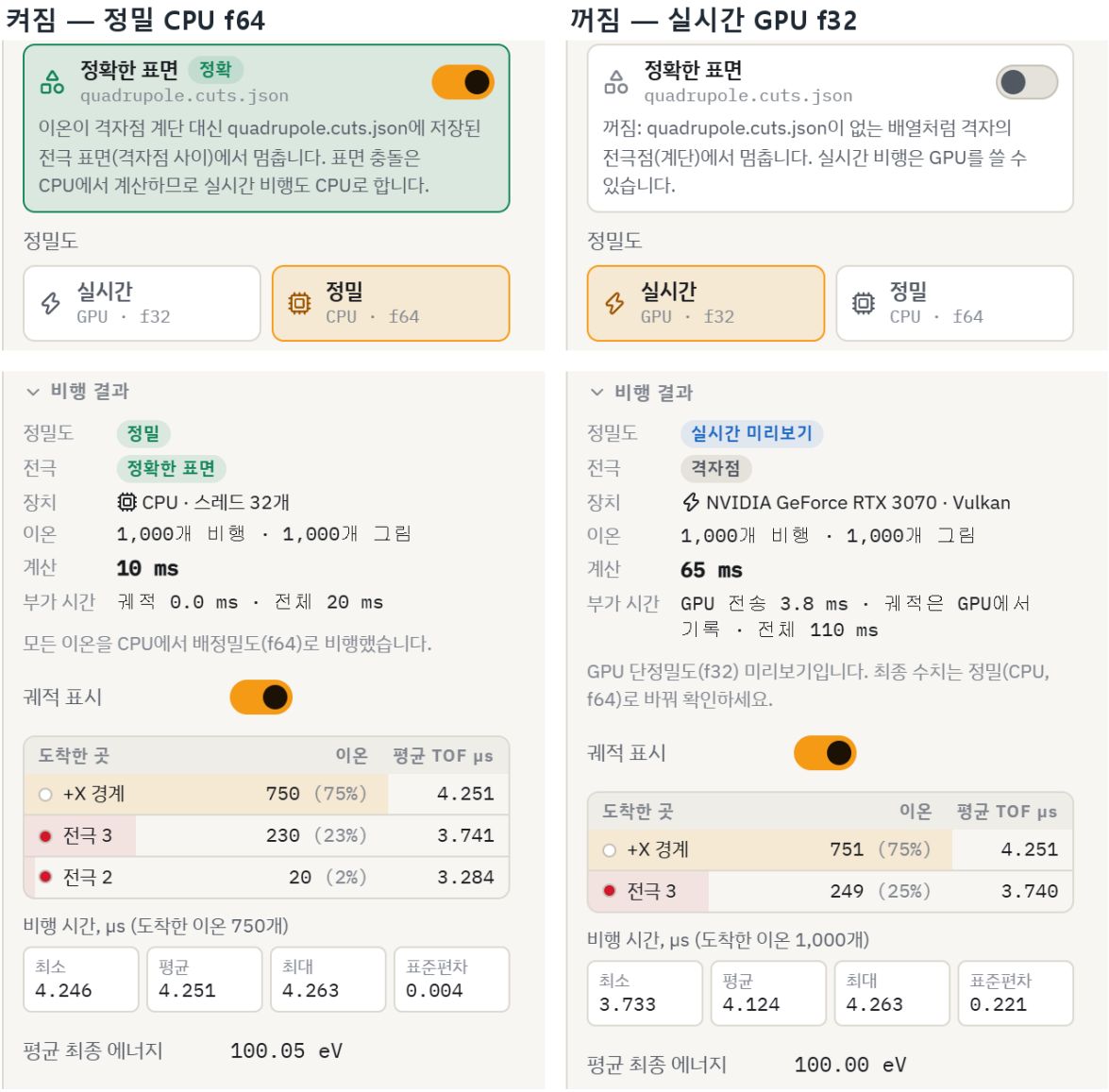


그림 21. 정확한 표면 카드와 결과 배지. 표면 파일 이름, 켜고 끄는 스위치, 비행에 쓴 표면 종류를 보여준다. 출처: figures/F22.png.

설치 파일은 관리자 권한 없이 사용자 계정에 깔리는 NSIS 패키지다. 크기는 28.5 MiB 이고 STEP 가져오기용 OpenCascade 라이브러리와 실행에 필요한 Visual C++ 런타임을 함께 담는다. 조용한 설치부터 실행, STEP 과 IGES 가져오기, 비행, 제거까지 실제로 확인했다(docs/dev/frontend.md 의 Windows 설치 패키지 절).

> **요점 정리**
>
> 앱은 템플릿에서 결과까지 한 화면 흐름으로 이어진다. 따라하기, 실시간 슬라이더, 정확한 표면 표시, 한국어 화면을 갖췄다. 설치에 관리자 권한이 필요 없다.

8. 종합 결과

> **이 장에서 배울 것**
>
> 앞 장의 비교를 한 표에 모아 SimRep 이 처음 목표를 얼마나 달성했는지 확인한다.

그림 22 는 각 장에서 정밀도를 맞춰 비교한 계산 시간 배수를 한 그림에 모았다. 가로축은 SIMION 계산 시간을 SimRep 계산 시간으로 나눈 값(로그축)이고 색은 기능 분야다. 모든 막대가 1 보다 오른쪽에 있으므로 비교한 모든 경우에서 SimRep 이 빨랐다.

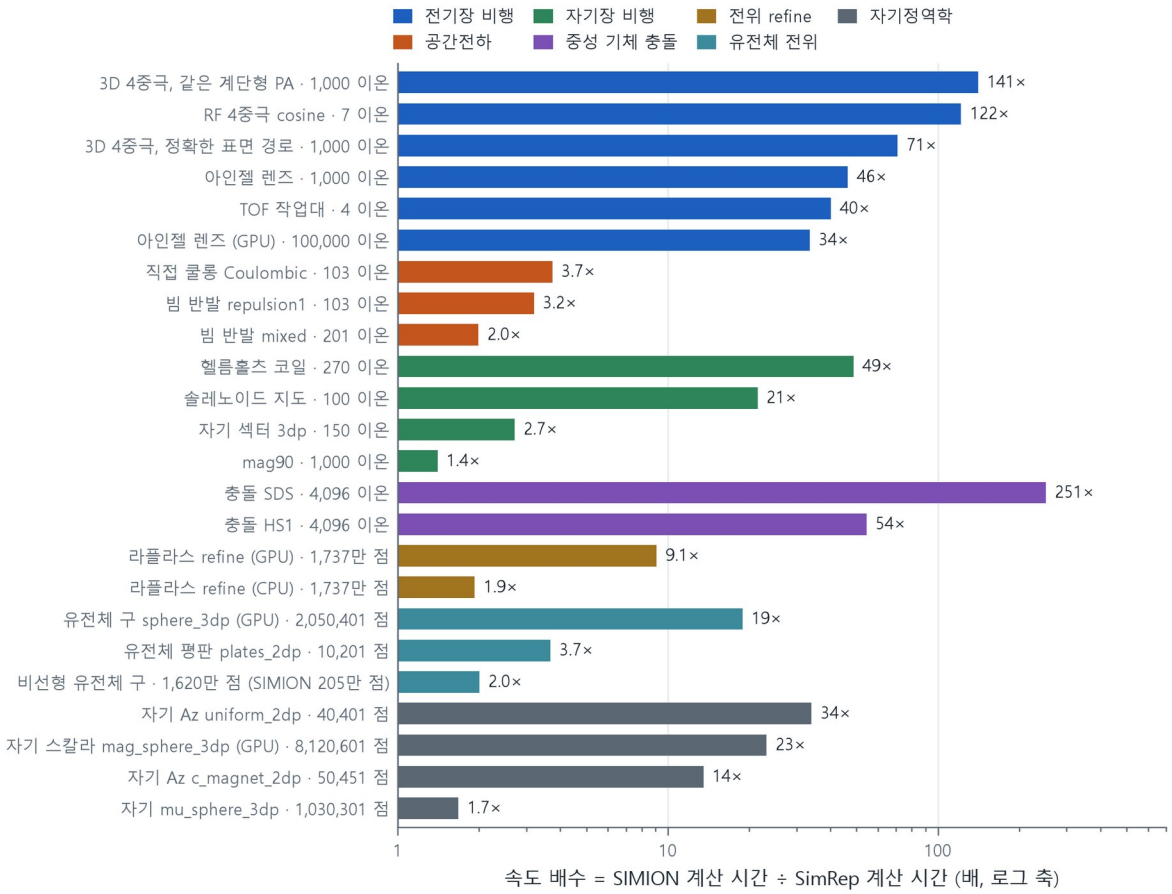


그림 22. 정밀도를 맞춘 비교에서 SIMION 대비 계산 시간 배수(로그축). 계산 시간에 넣은 범위는 분야마다 다르다. 전위 계산은 준비와 풀이, 비행은 비행 계산만, 유전체는 재료 준비·계층 구성·풀이를 합했다. 유전체 막대(3.7 배, 18.9 배)는 표 9 의 자동 선택 행이 아니라 잔차 1e-12 로 보존한 비교 설정이다. 각 막대의 두 계산 시간과 출처는 figure-ledger.json 의 F02 에 있다. 출처: figures/F02.png.

분야	대표 비교	정확도	계산 시간 배수
전위 계산	259^3 배열, GPU	엄격 SIMION 과 6.4e-5 V 이내	19.3 (준비 포함 9.1)
경계 처리	두 구 전극, 같은 메모리	오차 78~84 배 감소	—
비행	einzel 1 천 개,	RMS 0.226 대 1.106	46

	CPU(보존 one-cell 설정), T.Qual 209 대비	mm	
비행	3D 4 중극, 정확한 표면	RMS 0.0034 대 2.38 mm	71
자기장	SIMION 예제 7 개	기존 기준 통과	1.4~48.7
충돌	HS1, SDS 4,096 개	KS 검정 기각 없음	54, 251
유전체	선형 3 개, 비선형 1 개	엄격 설정·연속 극한 추정 기준	1.8~23.4
공간전하	Coulombic, beam 2 개	더 작은 오차	2.0~3.7
자기 포텐셜	계산 22 건	저장 포텐셜 기준 SIMION 기본 이상	1.7~34

표 13 의 값은 표 아래 문단에 행별로 적은 원본 문서 절에서 가져와 이 문서에서 모은 것이다. 행별 출처는 다음과 같다. 전위 계산(259³ 배열, GPU) 행은 bench/refine-real.md 의 End-to-end performance·Large-array tight reference 절, bench/simion-baseline.md 의 Refine 절; 경계 처리(두 구 전극, 같은 메모리) 행은 docs/cutcell.md 의 Equal memory budget 절; 비행(einzel 1 천 개, CPU(보존 one-cell 설정), T.Qual 209 대비) 행은 bench/default-flight.md 의 Retained one-cell baseline 절, bench/fly-real.md 의 Boundary-aware correction results 절; 비행(3D 4 중극, 정확한 표면) 행은 bench/fly-real.md 의 Compute time at achieved precision 절, docs/geometry-pipeline.md 의 V3 compute overhead and verification 절, bench/fly-real.md 의 Physical geometry and the better field route 절; 자기장(SIMION 예제 7 개) 행은 bench/magnetic-cpu.md 의 Actual matched-population compute·Unchanged accuracy gate 절; 충돌(HS1, SDS 4,096 개) 행은 bench/default-flight.md 의 HS1 and SDS statistical checks 절; 유전체(선형 3 개, 비선형 1 개) 행은 docs/dielectric.md 의 Automatic selection at the default residual tolerance·Speed at matched estimated physical accuracy 절; 공간전하(Coulombic, beam 2 개) 행은 bench/space-charge.md 의 Matched-precision compute verdict and defaults 절, bench/default-flight.md 의 Retained one-cell baseline 절; 자기 포텐셜(계산 22 건) 행은 docs/magnetostatics.md 의 Audit results·Hardware GPU and uniform-field follow-up·Uniform Az root cause and fix 절. 처음 계획의 단계 목표와 비교하면, 큰 3D 전위 계산은 엄격 SIMION 의 5 배 이상이라는 목표를 GPU 에서 넘었고, 비행은 정확도를 지키며 10 배 이상이라는 목표를 einzel 과 4 중극에서 넘었으며, 같은 메모리에서 오차 10 배 감소라는 목표도 넘었다(docs/PLAN.md 의 단계별 계획 절). 정확도는 이 보고서에서 고른 대표 비교마다 각 절의

판정 기준을 충족했다. 기준은 비교마다 다르다. 자기 포텐셜은 저장 포텐셜, 비선형 유전체는 연속 극한 추정 오차, 무작위 모델은 분포 검정이 기준이며, beam 모드는 SIMION 내부 구현과의 동등성까지 확인한 것은 아니다.

> **요점 정리**
>
> 이 보고서가 고른 대표 비교에서는 각 절의 정확도 기준을 충족하면서 SimRep 이 SIMION 보다 빨랐다(그림 22, 표 13). 계획한 세 단계 목표도 모두 넘었다(docs/PLAN.md 의 단계별 계획 절).

9. 한계와 남은 일

> **이 장에서 배울 것**
>
> 이 보고서의 결론이 어디까지 성립하는지와, 아직 하지 않은 일을 정리한다.

결론에는 다음 조건이 붙는다. 첫째, 정확도 기준해는 촘촘한 격자로 얻은 수치 기준이며 연속해의 엄밀한 상한이 아니다. 둘째, cut-cell 은 전위에서 2 차 정확도이지만 전극 표면 바로 옆 전기장은 약 1 차이고 곡면 유전체와 자성체 경계도 약 1 차다. 셋째, 유전체와 자기 포텐셜, 앱의 측정 일부는 다른 작업과 기계를 나눠 쓰는 동안 했다. 이 사실은 각 원본 문서에 표시했다. 넷째, 측정 장비는 RTX 3070 한 대이며 배정밀도가 빠른 GPU 에서는 CPU 와 GPU 의 선택 기준이 달라진다.

항목	현재 상태	근거
자기장 GPU 비행	구현됨(--device gpu). RTX 3070 에서는 CPU 가 3~9 배 빨라 자동 선택은 CPU. 성능 개선과 자동 기준 재검토가 남음	bench/fly-gpu.md 의 Magnetic fields and neutral gas 절
정확한 표면의 GPU 비행	CPU 전용	docs/geometry-pipeline.md 의 Outputs and flying 절
적응 격자(octree)	보류(사용자 결정). cut-cell 로 단계 목표를 넘음	docs/PLAN.md 의 단계별 계획 절(3 단계)
비선형 B-H 자성체, 위치마다 다른 투자율의 3D 벡터 포텐셜	미구현	docs/magnetostatics.md 의 Equations and flight contract 절
Lua 작업대 사용자 프로그램	지원하지 않음. 전압 수식과 설정으로 대신함	docs/PLAN.md 의 3.4 절

STEP 내보내기	미구현. 메시와 VTK 내보내기만 있음	docs/mesh-io.md 의 Export surfaces and fields 절
외부 자기장 지도	규칙 격자만 지원. 흩어진 점 데이터는 다시 내보내야 함	docs/bfield-import.md 의 Accepted data 절
GPU 하드웨어 계수(점유율, 레지스터)	측정하지 않음. Nsight Compute 는 CUDA 용이고, Vulkan 용 Nsight Graphics/Systems 는 설치·측정하지 않음	bench/fly-gpu.md 의 Phase replay and generated shader evidence 절

표 14 는 근거 열에 적은 각 문서 절의 한계를 이 문서에서 모은 것이다. 이 가운데 자기장 GPU 비행의 성능은 배정밀도 연산이 빠른 GPU 나 혼합 정밀도 방식으로 다시 볼 수 있고, 정확한 표면의 GPU 비행은 메시 교차 계산을 GPU 로 옮기면 된다.

> **요점 정리**
>
> 결론은 수치 기준해, RTX 3070 한 대, 일부 공유 기계 측정이라는 조건 위에서 성립한다. 자기장 GPU 비행의 성능 개선, 정확한 표면의 GPU 비행, STEP 내보내기, 비선형 자성체가 남은 일이다(표 14).

참고문헌

Gibou, F., Fedkiw, R. P., Cheng, L.-T., & Kang, M. (2002). A second-order-accurate symmetric discretization of the Poisson equation on irregular domains. Journal of Computational Physics, 176(1), 205–227. https://doi.org/10.1006/jcph.2001.6977

Jackson, J. D. (1999). Classical electrodynamics (3rd ed.). New York: Wiley.

Open Cascade SAS. (n.d.). STEP Translator. Open CASCADE Technology 8.0.1 documentation (online edition, accessed 2026-10-24; SimRep was built against OCCT 7.7.0). https://dev.opencascade.org/doc/overview/html/occt_user_guides__step.html

Scientific Instrument Services. (2024a). Magnetic potential [SIMION 2024 help page magnetic_potential.html]. SIMION 2024 documentation, installed with SIMION 2024.

Scientific Instrument Services. (2024b). Dielectrics [SIMION 2024 help page dielectric.html]. SIMION 2024 documentation, installed with SIMION 2024.

Scientific Instrument Services. (2024c). collision_hs1 example [examples/collision_hs1/collision_hs1.lua]. SIMION 2024 example files, installed with SIMION 2024.